Simulateur de Golf en Scilab : Capture d'un swing et estimation de la portée avec un petit radar low-cost

Matthieu PHILIPPE
Publication Perso

8 novembre 2026

Table des matières

1 Introduction 2

2 Vol de la balle de golf 4
2.1 Force de traînée . . . 5
2.1.1 Caractéristiques de l'air . . . 5
2.1.2 Caractéristiques de la balle dans l'air . . . 5
2.2 Effet Magnus . . . 6
2.3 Force de gravité . . . 7
2.4 Vitesse de départ . . . 8
2.5 Vitesse de la balle avec un centrage imparfait . . . 9
2.6 Vitesse initiale de la face du club . . . 9
2.7 Modèle complet de vol . . . 10
2.7.1 Modèle complet de vol (forme matricielle/vectorielle) . . . 10
2.7.1.1 Conditions initiales . . . 10
2.7.2 Modèle complet de vol (notation tensorielle) . . . 10
2.7.2.1 Équation fondamentale . . . 10
2.7.2.2 Force de traînée . . . 11
2.7.2.3 Force de Magnus . . . 11
2.7.2.4 Force de gravité . . . 11
2.7.2.5 Initial Conditions . . . 11
2.8 Fonctions externes pour le solveur d'équations différentielles ordinaires . . . 11
2.8.1 Architecture et conception . . . 12
2.8.2 Fonctions de définition des équations . . . 12
2.8.3 Détection d'événements . . . 12
2.8.4 Intégration numérique . . . 13
2.8.5 Limitations et évolutions potentielles . . . 14

3 Rebonds et roulement 15
3.1 Rebonds et roulement . . . 15
3.1.1 Caractéristiques de la balle pendant les rebonds . . . 15
3.1.1.1 Angle d'impact et restitution . . . 15
3.1.1.2 Angle de rebond et mise à jour des vitesses . . . 16
3.1.1.3 Frottement et spin pendant le rebond . . . 16
3.1.2 Caractéristiques de la balle pendant le roulement . . . 17
3.1.2.1 Résistance au roulement . . . 17
3.1.2.2 Condition d'arrêt . . . 18
3.1.3 Limitations et évolutions potentielles . . . 19
3.2 Explication du code Scilab de simulation de la balle de golf . . . 20
3.2.1 La fonction `flightg` . . . 20
3.2.1.1 Entrées et sorties . . . 20
3.2.1.2 Logique interne . . . 20
3.2.2 La fonction `rolling` . . . 21
3.2.2.1 Entrées et sorties . . . 22

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

3.2.2.2 Internal Logic 22
3.2.3 Les fonctions AltiSol et dAltiSol 22
3.2.3.1 La fonction AltiSol 23
3.2.3.2 La fonction dAltiSol 23
3.2.4 Fonctions de détection d'événements 23
3.2.4.1 La fonction events 23
3.2.4.2 La fonction events2 23
3.2.5 Intégration numérique avec ode 24
3.2.5.1 Appel de la fonction ode 24

4 Matériel 25

5 Analyse des données 28
5.1 Captures radar filtrées 28
5.1.1 Filtrage 29
5.1.2 Résultat final 30

6 Pipe de traitement du signal radar Doppler : de Scilab à Python 32
6.1 Géométrie des deux radars et observables Doppler 32
6.2 STFT et suivi du ridge 33
6.3 Vitesse : médiane du ridge post-impact vs max global 33
6.4 Spin : périodicité du patch vs excursion spectrale 34
6.5 Appariement stéréo des deux ridges 34
6.6 Contrôles qualité (« non identifiable » est une sortie valide) 35
6.7 Scilab → Python : corrections et résultats de validation 35
6.8 Pipe de traitement 36
6.9 Spectrogrammes : captures mesurées vs modèle théorique 36

7 Listings 41
7.1 Golfball.sci 41

Table des figures

1.1 IHM de la simulation 2
1.2 Captures radar 3

2.1 **Schéma des forces agissant sur une balle de golf en vol.** 4
2.2 Composantes de vitesse 8
2.3 Trajectoire du club 9

4.1 La valise 25
4.2 Intérieur avec Arduino, radar, amplificateur, cartes audio 26
4.3 Tiny Arduino 26
4.4 Carte audio 26
4.5 Carte amplificateur audio 27
4.6 Carte radar 27

5.1 Captures WAV radar et leur écho 28

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

5.2 WAV brut capturé . . . 29
5.3 FFT du brut filtré . . . 29
5.4 Résultats de mon algorithme de densité de puissance pour identifier vitesse et backspin . . . 30
5.5 Résultats de la densité spectrale de puissance filtrée en moyenne et normalisée . . . 30
5.6 Résultats du filtrage de la densité et de l'identification des caractéristiques . . . 31
5.7 Échos de vitesse et de backspin . . . 31

6.1 Vue de dessus du dispositif à deux radars. f_C porte V_x (vitesse), f_Δ porte V_z (azimut α dans le plan x–z). Le facteur $2L/b$ corrige la parallaxe : plus la balle est loin, plus la différence de fréquence est faible pour le même V_z. . . . 33
6.2 Le patch aluminium comme diffuseur dominant unique. La périodicité du flash comme l'excursion spectrale mesurent le spin *total* ω ; aucune des deux ne sépare backspin et sidespin. . . . 34
6.3 Pipe de validation Python (`sci/pipe_validation.py`). Équivalents Scilab : `animDensite` (STFT), `Info/GetSwing` (vitesse), `animHilbert` (enveloppe), `acos(f2/f1)` (azimut rudimentaire dans `testsFFT.sci`). . . . 36
6.4 Géométrie du dispositif radar (aussi dans le README). . . . 36
6.5 Patch sur la balle (ruban aluminium). . . . 37
6.6 Spectrogramme mesuré, coup 123 m / 150 km/h / 5274 tr/min (référence SkyTrak). . . . 37
6.7 Spectrogramme mesuré, coup 131 m / 162 km/h / 6185 back + 7102 latéral tr/min ($\beta \approx 48{,}9°$, forte courbure). . . . 38
6.8 Spectrogramme mesuré, coup 137 m / 166 km/h / 4990+1312 tr/min ($\beta \approx 14{,}7°$). . . . 38
6.9 Capture théorique (générée), config F7 151 km/h, départ 16,5°, spin 4000 tr/min, latéral 700 tr/min ($\beta = 10{,}1°$), club 115 km/h, portée 102 m. . . . 39
6.10 Capture générée, même config, version swing complet 450 ms (v2). . . . 39
6.11 Capture générée avec les paramètres du vrai coup de 137 m (166 km/h, 4990/1312 tr/min, club 118). . . . 40
6.12 Trajectoire modèle (vol + déviation latérale du slice) pour la capture générée du 2025-11-08. . . . 40

Liste des tableaux

2.1 Table de mes C_{l1} . . . 7

6.1 Résultats mesurés, 11 CSV (9 vrais coups F7 + 2 signaux modèle). . . . 35

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

Listings

3.1 Appel de la fonction ode pour la phase de vol 24
7.1 Listing de la simulation de vol de la balle de golf 41

Liste des équations

2.1 Forces en vol 4
2.2 Résistance de l'air 5
2.2 Masse, diamètre, rayon, inertie, section, nombre de Reynolds 5
2.5 Coefficient de traînée, force de traînée 5
2.7 Effet de portance Magnus 6
2.12 Effet Magnus et différents modèles de coefficient 7
2.13 Angle entre axes de mouvement et de spin 7
2.16 Force de gravité 7
2.21 Vitesse de la balle et angle de départ après l'impact du club 8
2.22 Vitesse initiale de la balle après un centrage imparfait 9

Liste des remarques

1 Enfin, l'équation de la force de traînée selon les 3 axes devient 2.6 6
2 Connaissant 2.7, 2.14 et 2.1, j'ai choisi le modèle de force de Magnus suivant 7
3 La force de gravité 7

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

Liste des définitions

1 Force de traînée de l'air 5
2 Effet Magnus 6
3 La force de gravité 7

RÉSUMÉ

En utilisant les équations de la dynamique de vol d'une balle de golf, cet article montre comment coder un simulateur de swing de balle en Scilab.

Le simulateur utilise un solveur d'EDO (équations différentielles ordinaires) de Scilab, les équations de la dynamique des fluides appliquées à une balle (gravité, traînée, forces de Magnus), la restitution des forces d'un club de golf appliquée à une balle pour les départs, son vol, ses rebonds et son roulement.

Dans la seconde partie, l'article présente une méthode pour capturer la vitesse de la balle en vol à l'aide de 2 radars low-cost, couplés à un Arduino Mini et une carte son adaptée connectée à la sortie du radar.

Le signal radar est enregistré au format WAV puis filtré pour extraire vitesse et spin pendant les premiers mètres de vol, en utilisant la densité spectrale de puissance en mode temps réel.

Mots-clés : *Balle de golf, Simulateur, Scilab, Radar, Densité spectrale.*

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 1

Introduction

Après quelques années à jouer au golf, j'ai voulu en savoir plus sur la dynamique des interactions entre le club, la balle et les éléments.

Bien sûr, la littérature académique et les publications étaient assez abondantes. Mais je suis tombé sur des articles [**?**] de 1976 et [**?**] de 1994. La description de la physique de la balle m'a poussé à modéliser ce système en Scilab (fig 1.1). Ensuite, je l'ai fait en Java, C++, puis C# pour l'utiliser sous *Unity game engine*®™.

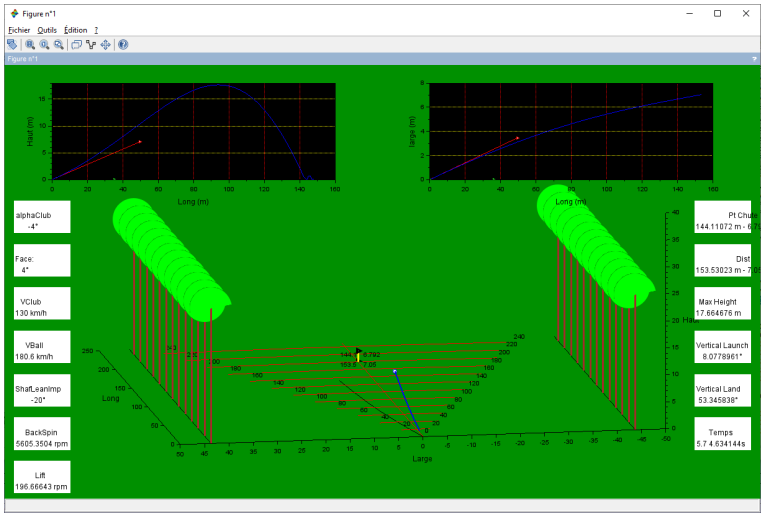


Figure 1.1 – IHM de la simulation

Quand les résultats ont été satisfaisants, j'ai pensé à capturer le vol d'une balle de golf avec de l'instrumentation. Après des essais au sonar, largement insuffisants, j'ai pris de petits radars de détection de présence pour portes automatiques.

Je ne savais pas dans quoi je m'engageais, mais après avoir remis à jour mes connaissances en filtrage du signal et quelques lignes de programmation en Scilab, j'ai trouvé un moyen de capturer les données essentielles : la vitesse et le backspin de la balle en vol, ainsi que l'écho du club juste avant l'impact et pendant le vol (fig 1.2).

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

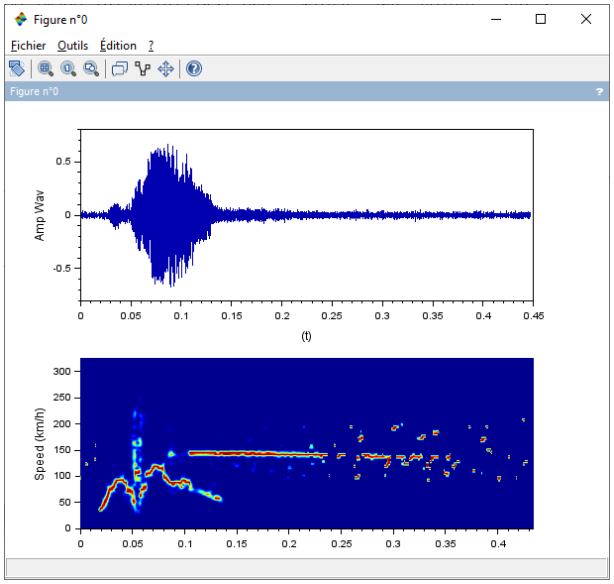


FIGURE 1.2 – Captures radar

Cet article décrit tout ce que j'ai trouvé et comment j'ai assemblé le tout : équations, programmes, filtrage, estimateurs et composants électroniques pour réaliser une simulation virtuelle de golf.

Toutes les équations présentées dans cet article sont issues de publications ou d'ouvrages publics, référencés dans la bibliographie. Je ne reviendrai pas sur les démonstrations et explications physiques, qui ne sont pas l'objet de ces pages. Ce qui m'importe, c'est le lien entre ces informations et la façon dont je les ai utilisées ou adaptées.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 2

Vol de la balle de golf

L'équation principale est celle du vol de la balle. Elle rend compte de 3 forces : la force de Magnus, la traînée de l'air et la gravité, telles que :

$$\sum \overrightarrow{F} = \overrightarrow{F_m} + \overrightarrow{F_d} + \overrightarrow{F_g} \tag{2.1}$$

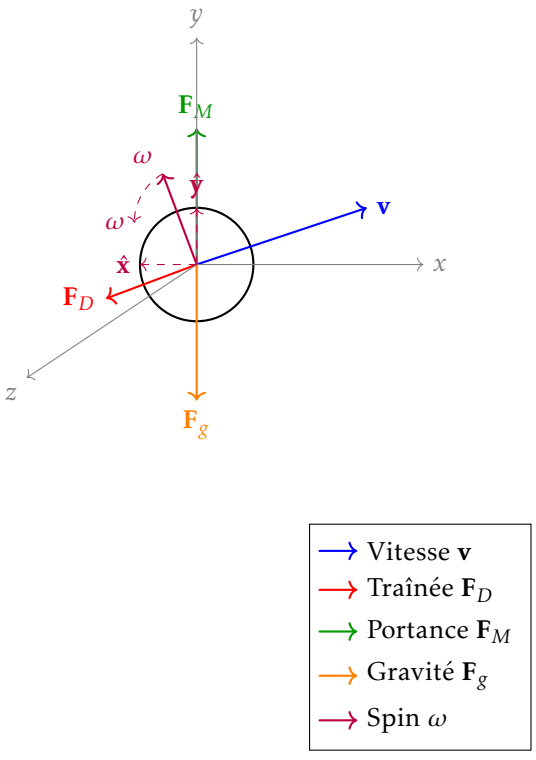


FIGURE 2.1 – **Schéma des forces agissant sur une balle de golf en vol.**

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

2.1 Force de traînée

2.1.1 Caractéristiques de l'air

Définition 1: Force de traînée de l'air	*La force de traînée de l'air est la résistance s'exerçant sur la section droite d'un objet, orthogonale au déplacement, déterminée par la relation donnée par [?] (2.2)* :

$$F_d(V) = -\frac{1}{2}.\rho(T).C_d.S.V^2(t) \tag{2.2}$$

où :

$$\begin{aligned}
\rho(T) &= 1,292 * \frac{273,15}{273,15+T} \\
\rho(T) &= densit\ de\ l'air\ \rho\ selon\ la\ temprature\ T \\
T &= Temprature \\
C_d &= Coef\,ficient\ de\ trane,\ dpendant\ de\ l'objet \\
V &= Vitesse\ de\ l'objet \\
F_d(V) &= Force\ de\ trane\ due\ \ la\ rsistance\ de\ l'air,\ selon\ la\ vitesse\ V \\
S &= Section\ droite\ expose,\ orthogonale\ au\ dplacement.
\end{aligned}$$

2.1.2 Caractéristiques de la balle dans l'air

En considérant ces caractéristiques d'un modèle de balle alvéolée,

$$\begin{aligned}
mass\ m &= 0,04593\ g \\
Dynamic\ viscosity\ \mu &= 1,5.10^{-6} m^2/s \\
Diameter\ D &= 0,04267\ m \\
Radius\ r &= \frac{D}{2} \\
Inertia\ I &= \frac{2}{5}.m.R^2 \\
Cross-section\ S &= \pi.R^2 \\
Ball\ Velocity &= V
\end{aligned}$$

Nous avons un modèle estimé,

$$Re = \frac{\rho(T).V.D}{\mu} \tag{2.3}$$

$$C_d = 0,36 + \frac{24}{Re} + \frac{6}{1+\sqrt{Re}} \tag{2.4}$$

$$\overrightarrow{F_d} = -\frac{1}{2}.\rho(T).C_d.S.\sqrt{(V(x)^2+V(y)^2+V(z)^2)}.\widehat{V} \tag{2.5}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

où :

$$\begin{aligned} \widehat{V} &= Vecteur\ unitaire\ de\ la\ vitesse \\ Re &= Estimateur\ du\ nombre\ de\ Reynolds \\ C_d &= Coefficient\ de\ trane,\ relation\ de\ White \\ \overrightarrow{F_d} &= Force\ de\ trane \end{aligned}$$

Remarque 1: *Enfin, l'équation de la force de traînée selon les 3 axes devient 2.6*	$\overrightarrow{F_d} = \begin{pmatrix} -\frac{1}{2}.\rho(T).C_d.S.v_i^2 \\ -\frac{1}{2}.\rho(T).C_d.S.v_j^2 \\ -\frac{1}{2}.\rho(T).C_d.S.v_k^2 \end{pmatrix}$ (2.6)

2.2 Effet Magnus

Définition 2: Effet Magnus	*L'effet Magnus [?] est dû à la rotation d'un objet cylindrique se déplaçant dans un fluide (l'air). On l'appelle effet Robin pour une balle.*

Par généralisation, le nom d'effet Magnus est appliqué à tout objet soumis à cette force d'aspiration. Cette force de portance apparaît et modifie la trajectoire balistique de la balle de golf. La force dépend du coefficient de portance (C_l), associé à l'objet selon sa forme, sa nature et sa texture. La force s'écrit donc comme une composante de la vitesse de l'objet, du taux de spin et d'un coefficient d'effet Magnus C_m (2.7),

$$\overrightarrow{F_m} = C_m.(\overrightarrow{\omega} \times \overrightarrow{V}) \quad (2.7)$$

où :

$$\begin{aligned} \omega &= spin\ en\ rad/s \\ V_p &= r\omega\ vitesse\ priphrique \\ V &= Vitesse\ de\ la\ balle \\ C_m &= Coefficient\ d'effet\ Magnus \end{aligned}$$

La bibliographie principale que j'ai utilisée pour la balle de golf est [?] et [?]. J'ai trouvé plusieurs façons de calculer le C_m, données ci-dessous. La première approximation simple est donnée par 2.8, et le coefficient d'effet Magnus peut être donné par les deux premiers estimateurs ci-dessous 2.9, 2.10 ([?]). Mais j'ai retenu celui-ci 2.11 ([?], [?]).

$$F_m = \frac{1}{2}.C_m.\rho.\mu^2 \quad (2.8)$$

$$C_m = \frac{r\omega}{V} \quad (2.9)$$

$$C_m = -0,05 + \sqrt{0,0025 + 0,36\left(\frac{r\omega}{V}\right)} \quad (2.10)$$

$$C_m = 0,5.\rho(T).S.R^2.C_l.\left|\frac{\omega}{V}\right|^{0,4}.V \quad (2.11)$$

$$C_l = \frac{1}{2 + (V/R\omega) * sin(\zeta)} \quad (2.12)$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

[?] donne un coefficient de correction C_l (2.12) à appliquer à (2.11), où ζ, dans le coefficient de portance, est l'angle entre l'axe de rotation et la direction du mouvement. Si R$\omega \ll$ V, alors $C_l \simeq R\omega\frac{sin\zeta}{V}$. Certainement plus approprié que (2.9) pour prendre en compte l'effet du spin latéral et du backspin.

$$sin\zeta = \frac{\| \overrightarrow{\omega} \times \overrightarrow{V} \|}{\omega.V} = \frac{1}{\omega.V}.\sqrt{((\omega_j.V_k - \omega_k.V_j)^2 + (\omega_k.V_i - \omega_i.V_k)^2 + (\omega_i.V_j - \omega_j.V_i)^2)} \tag{2.13}$$

Pour répondre à mes besoins, j'ai choisi de fournir mes valeurs adaptées de C_l à (2.11) plutôt que (2.12), issues de tests avec mes propres balles (la texture et le nombre d'alvéoles jouent sur le nombre de Reynolds (2.3) comme montré dans [?]). Ces données sont enregistrées dans une table Scilab C_{l1}. C_m devient (2.14).

$$C_m = 0,5.\rho(T).S.R^2.C_{l1}.|\frac{\omega}{V}|^{0,4}.V \tag{2.14}$$

Mes propres C_{l1} pour les clubs Mizuno JPX et la balle Decathlon sont données en exemple dans le tableau 2.1.

Club	C_{l1}
D	0.64
W5	0.65
H3	0.54
I5	0.54
I6	0.83
I7	0.42
I8	0.53
I9	0.53
PW	0.52
AW	0.52
SW	0.51
LW	0.51

TABLE 2.1 – Table de mes C_{l1}

Remarque 2: Connaissant 2.7, 2.14 et 2.1, j'ai choisi le modèle de force de Magnus suivant	$$\overrightarrow{F_m} = \begin{pmatrix} C_m * (\omega_j * v_k - \omega_k * v_j) \\ C_m * (\omega_k * v_i - \omega_i * v_k) \\ C_m * (\omega_i * v_j - \omega_j * v_i) \end{pmatrix} \tag{2.15}$$

2.3 Force de gravité

Définition 3: La force de gravité	*La force de gravité est déterminée par l'accélération g vers le sol.*

L'expression de la force de gravité est simplement,

Remarque 3: La force de gravité	$$\overrightarrow{F_g} = \begin{pmatrix} 0 \\ -mg \\ 0 \end{pmatrix} \tag{2.16}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.4 Vitesse de départ

Par l'impact du club de golf sur la balle, il résulte un vol de balle avec un angle de départ. Le loft de la tête de club détermine l'angle de départ ; la vitesse de la tête de club et la texture des deux déterminent la vitesse de balle au départ et le spin de la balle [**?**].

À cause de la texture, il y a une perte d'énergie, que l'on peut prendre en compte via le coefficient de restitution e (COR [**?**]).

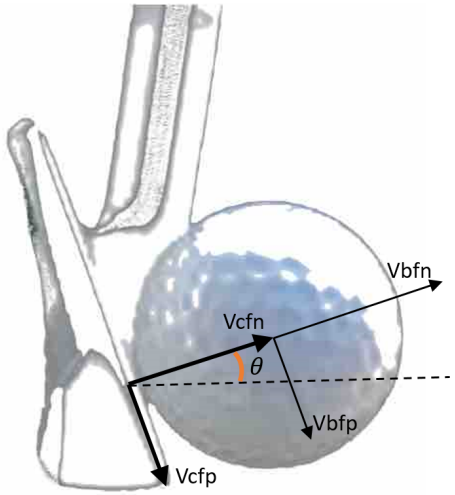


FIGURE 2.2 – Composantes de vitesse

[**?**] donne une estimation du COR e :

$$e = 0,86 - 0,0029.V_{ci}.cos(\theta) \tag{2.17}$$

$$V_{bfn} = V_{ci}.cos(\theta).\frac{1+e}{1+\frac{m}{M}} \tag{2.18}$$

$$V_{bfp} = -V_{ci}.\frac{sin(\theta)}{1+\frac{m}{M}+\frac{5}{2}} \tag{2.19}$$

$$\psi_{bo} = \theta + \arctan\left(\frac{V_{bfp}}{V_{bfn}}\right) \tag{2.20}$$

$$\omega_{bo} = \omega_{bf} = -m.V_{bfp}.\frac{R}{I} \tag{2.21}$$

où :

V_{ci} = $Vitesse\ initiale\ du\ club,\ parallle\ au\ sol.$
V_{bfn} = $Vitesse\ finale\ de\ la\ balle\ orthogonale\ \ la\ face\ du\ club$
V_{bfp} = $Vitesse\ de\ la\ balle\ de\ golf,\ parallle\ \ la\ face\ du\ club$
θ = $loft\ dynamique\ (\ loft\ du\ club + inclinaison\ du\ shaft\ \ l'impact\ <\ loft)$
e = $COR,\ e \simeq 0,78\ pour\ les\ fers,\ e \simeq 0,83\ pour\ les\ drivers\ et\ les\ bois$
ψ_{bo} = $Angle\ de\ dpart$
ω_{bo} = $Spin\ de\ la\ balle\ de\ golf$
I = $Inertie\ (2.3)$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.5 Vitesse de la balle avec un centrage imparfait

Il est difficile de centrer parfaitement la balle et le club. Le modèle de vitesse de balle est modifié pour inclure un paramètre *Miss*. C'est l'écart au sweet spot ; la vitesse initiale de la balle dans ce cas est (2.22),

$$V_{bi} = (1 - 35.56 * miss).\sqrt{V_{bfn}^2 + V_{bfp}^2}\ (miss\ mesur\ en\ mtre) \tag{2.22}$$

2.6 Vitesse initiale de la face du club

En fonction de la trajectoire du club (α_{cp}) et de l'angle face/trajectoire (γ_{fp}), la vitesse initiale et le spin de la balle sont modifiés.

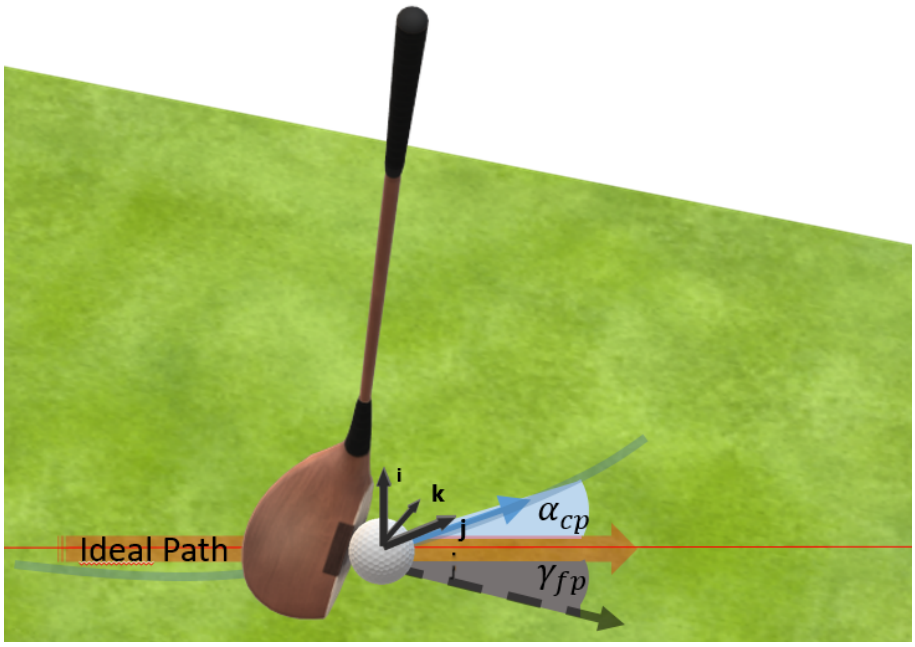


Figure 2.3 – Trajectoire du club

Ainsi :

$$V_{b0} \cdot \mathbf{i} = V_{bi} \cdot (1 - 0.3556 \cdot \text{miss}) \cdot \cos\left(\theta + \arctan\left(\frac{\sin(\theta)\cdot(1+\frac{m}{M})}{(1+e)\cdot\cos(\theta)\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right)$$

$$V_{b0} \cdot \vec{j} = V_{bi} \cdot (1 - 0.3556 \cdot \text{miss}) \cdot \sin\left(\theta + \arctan\left(\frac{\sin(\theta)\cdot(1+\frac{m}{M})}{(1+e)\cdot\cos(\theta)\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right)$$

$$V_{b0} \cdot \vec{k} = 0$$

Le spin autour de l'axe $\vec{i}$ n'est pas une composante le long de l'axe de la trajectoire du club. Le spin s'applique dans le plan du sol, avec le *SideSpin* aligné sur l'axe de la face du club $\vec{j}$, et le *BackSpin* aligné sur l'axe de la trajectoire du club $\vec{k}$. $w_{\vec{i}} = 0$

$$w_{\vec{j}} = V_{b0} \cdot \vec{j} \cdot \sin\left(\alpha_{cp} - \gamma_{fp}\right) \cdot \text{Sac.coeffSpinLift(Sac.Type == Club)}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

$w_{\vec{k}} = V_{b0} \cdot \vec{i} \cdot \sin(\theta) \cdot \text{Sac.coeffSpin(Sac.Type == Club)}$

2.7 Modèle complet de vol

2.7.1 Modèle complet de vol (forme matricielle/vectorielle)

Le vol de la balle de golf est décrit par le système d'EDO suivant pour le vecteur d'état $\vec{Y} = \begin{pmatrix} x & y & z & V_x & V_y & V_z \end{pmatrix}^T$:

$$\frac{d\vec{Y}}{dt} = \begin{pmatrix} V_x \\ V_y \\ V_z \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_x + \frac{C_m}{m}(\omega_y V_z - \omega_z V_y) \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_y + \frac{C_m}{m}(\omega_z V_x - \omega_x V_z) - g \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_z + \frac{C_m}{m}(\omega_x V_y - \omega_y V_x) \end{pmatrix} \tag{2.23}$$

où :

- $\vec{V} = \begin{pmatrix} V_x \\ V_y \\ V_z \end{pmatrix}$ est le vecteur vitesse,
- $\vec{\omega} = \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}$ est le vecteur spin,
- $\|\vec{V}\| = \sqrt{V_x^2 + V_y^2 + V_z^2}$ (norme),
- C_d est le coefficient de traînée (équation 2.4),
- $C_m = 0.5\rho(T)SR^2C_{l1}\left(\frac{\|\vec{\omega}\|}{\|\vec{V}\|}\right)^{0.4}\|\vec{V}\|$ (équation 2.14).

2.7.1.1 Conditions initiales

Le vecteur d'état initial $\vec{Y}_0$ est :

$$\vec{Y}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ V_{bi}\cos\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ V_{bi}\sin\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ 0 \end{pmatrix} \tag{2.24}$$

avec $V_{bi} = (1-35.56\cdot\text{miss})\sqrt{V_{bfn}^2 + V_{bfp}^2}$ et $\vec{\omega}_0 = \begin{pmatrix} 0 & V_{b0y}\sin\left(\alpha_{cp} - \gamma_{fp}\right)\cdot\text{SpinLiftCoeff} & V_{b0x}\sin(\theta)\cdot\text{SpinCoeff} \end{pmatrix}^T$.

2.7.2 Modèle complet de vol (notation tensorielle)

La dynamique de vol de la balle de golf est décrite en notation tensorielle comme suit :

2.7.2.1 Équation fondamentale

L'évolution temporelle du vecteur d'état $\vec{Y} = \begin{pmatrix} \vec{r} \\ \vec{V} \end{pmatrix} \in \mathbb{R}^6$ est donnée par :

$$\frac{d\vec{Y}}{dt} = \begin{pmatrix} \vec{V} \\ \frac{1}{m}\left(\vec{F}_d + \vec{F}_m + \vec{F}_g\right) \end{pmatrix} \tag{2.25}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.7.2.2 Force de traînée

La force de traînée est un champ vectoriel non linéaire :

$$\vec{F}_d = -\frac{1}{2}\rho(T)\, C_d(\|\vec{V}\|, Re)\, S\, \|\vec{V}\|\, \vec{V}, \quad \|\vec{V}\| = \sqrt{\vec{V}\cdot\vec{V}} \tag{2.26}$$

où le coefficient de traînée C_d dépend du nombre de Reynolds $Re = \frac{\rho(T)\|\vec{V}\|D}{\mu}$:

$$C_d = 0.36 + \frac{24}{Re} + \frac{6}{1+\sqrt{Re}}. \tag{2.27}$$

2.7.2.3 Force de Magnus

La force de Magnus s'exprime à l'aide du produit vectoriel :

$$\vec{F}_m = C_m(\vec{\omega}\times\vec{V}), \quad C_m = \frac{1}{2}\rho(T)\, S\, R^2\, C_{l1}\left(\frac{\|\vec{\omega}\|}{\|\vec{V}\|}\right)^{0.4}\|\vec{V}\| \tag{2.28}$$

où $\vec{\omega}\times\vec{V}$ est le produit vectoriel antisymétrique, représentable via le symbole de Levi-Civita ϵ_{ijk} :

$$(\vec{\omega}\times\vec{V})_i = \sum_{j,k}\epsilon_{ijk}\,\omega_j\, V_k. \tag{2.29}$$

2.7.2.4 Force de gravité

$$\vec{F}_g = -mg\,\hat{e}_y, \quad \hat{e}_y = \begin{pmatrix}0\\1\\0\end{pmatrix}. \tag{2.30}$$

2.7.2.5 Initial Conditions

Le vecteur d'état initial $\vec{Y}_0$ est :

$$\vec{Y}_0 = \begin{pmatrix}\vec{0}\\ \vec{V}_{b0}\end{pmatrix}, \quad \vec{V}_{b0} = V_{bi}\,\hat{u}(\theta, e, m, M) \tag{2.31}$$

où $V_{bi} = (1 - 35.56\cdot\text{miss})\sqrt{V_{bfn}^2 + V_{bfp}^2}$ et le vecteur unitaire $\hat{u}$ vaut :

$$\hat{u} = \begin{pmatrix}\cos\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right)\\ \sin\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right)\\ 0\end{pmatrix}. \tag{2.32}$$

Le vecteur spin initial $\vec{\omega}_0$ est :

$$\vec{\omega}_0 = \begin{pmatrix}0\\ V_{b0y}\sin\left(\alpha_{cp} - \gamma_{fp}\right)\cdot\text{SpinLiftCoeff}\\ V_{b0x}\sin(\theta)\cdot\text{SpinCoeff}\end{pmatrix}. \tag{2.33}$$

2.8 Fonctions externes pour le solveur d'équations différentielles ordinaires

La simulation de la trajectoire de la balle de golf repose sur la résolution d'un système d'**équations différentielles ordinaires (EDO)** modélisant les forces physiques agissant sur la balle pendant le vol, les rebonds et le roulement. Cette section détaille la conception et l'implémentation des fonctions externes utilisées pour définir ces EDO, ainsi que leur intégration au solveur numérique. L'approche s'appuie sur la modularité pour séparer la modélisation physique de la résolution numérique, permettant flexibilité et maintenabilité.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.8.1 Architecture et conception

L'architecture du solveur d'EDO s'articule autour de **trois composants centraux** :

- **Définition des équations** : les fonctions externes (ex. `flightg` pour le vol et `rolling` pour le roulement) définissent le système d'EDO à partir des lois physiques.
- **Détection d'événements** : les fonctions comme `events` et `events2` gèrent les événements discrets (ex. impact de la balle au sol ou condition d'arrêt).
- **Intégration numérique** : la fonction `ode` de Scilab (ou équivalent dans d'autres langages) réalise l'intégration temporelle selon une méthode spécifiée (ex. Runge-Kutta ordre 4).

Cette séparation garantit que le modèle physique peut être modifié indépendamment du solveur numérique, ce qui est essentiel pour un développement et une validation itératifs.

2.8.2 Fonctions de définition des équations

Les EDO sont définies dans des fonctions externes qui retournent les dérivées temporelles des variables d'état. Par exemple, la fonction `flightg` en Scilab calcule les dérivées pour la phase de vol, en intégrant les forces suivantes :

- **Force de traînée** : la force de traînée est modélisée par la **formule de White** pour le coefficient de traînée C_d, qui dépend du nombre de Reynolds Re :
$$C_d = 0.36 + \frac{24}{\mathrm{Re}} + \frac{6}{1+\sqrt{\mathrm{Re}}}, \quad \text{où} \quad \mathrm{Re} = \frac{V \cdot D}{\nu}$$
Ici, V est la vitesse de la balle, D son diamètre, et ν la viscosité cinématique de l'air ($1{,}5 \times 10^{-5}\,\mathrm{m}^2/\mathrm{s}$).
- **Force de Magnus** : la force de Magnus, responsable de la portance due au spin, est calculée avec le **modèle de Smits et Smith** [**?**] :
$$C_m = C_{l1} \cdot r \cdot \left|\frac{\omega}{V}\right|^{0.4}$$
où C_{l1} est un coefficient empirique, r le rayon de la balle, ω le taux de spin et V la vitesse de la balle. La force de Magnus vaut alors :
$$\mathbf{F}_m = \frac{1}{2}\rho C_m A r(\boldsymbol{\omega} \times \mathbf{v})$$
où ρ est la masse volumique de l'air, A la section droite de la balle et $\mathbf{v}$ le vecteur vitesse.
- **Gravité** : la force de gravitation est simplement $\mathbf{F}_g = (0, 0, -mg)$, où g est l'accélération de la pesanteur.

Le vecteur d'état pour la phase de vol comprend la position (x, y, z), la vitesse (v_x, v_y, v_z) et le spin $(\omega_x, \omega_y, \omega_z)$ de la balle. Les EDO forment donc un système de 9 équations couplées :

$$\frac{d}{dt}\begin{bmatrix} x \\ v_x \\ y \\ v_y \\ z \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} v_x \\ \frac{F_{dx}+F_{mx}}{m} \\ v_y \\ \frac{F_{dy}+F_{my}}{m} - g \\ v_z \\ \frac{F_{dz}+F_{mz}}{m} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Notons que l'implémentation actuelle suppose **aucune décroissance du spin** pendant le vol ($\frac{d\omega}{dt} = 0$), ce qui est une simplification pouvant introduire des erreurs sur les longues distances (voir section 2.8.5).

2.8.3 Détection d'événements

Les événements discrets, comme l'impact de la balle au sol ou sa condition d'arrêt, sont gérés par des **fonctions d'événements**. Ces fonctions sont essentielles pour :

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

— Détecter quand la balle touche le sol ($y \leq \text{Altitude}(x, z)$).
— Déclencher la transition du vol vers les phases de rebond ou de roulement.
— Garantir la stabilité numérique en adaptant le pas de temps près des points d'événement.

En Scilab, la fonction `events` vérifie l'impact de la balle au sol :

```
function [value] = events(t, x)
    if (x(3) < AltiSol(x(1), x(5))) then
        value = 0;
    else
        value = x(3) - AltiSol(x(1), x(5));
    end
endfunction
```

Ici, `x(3)` représente la hauteur y de la balle, et `AltiSol` calcule l'altitude du sol à la position (x, z). L'événement est déclenché quand la hauteur de la balle passe sous le niveau du sol.

De même, la fonction `events2` détecte l'arrêt du roulement en vérifiant si la vitesse horizontale v_x passe sous un seuil :

```
function [value] = events2(t, x)
    value = x(2) - 0; // Arret si v_x <= 0
endfunction
```

Limitations :
— La détection d'événements actuelle utilise un **seuil fixe** (ex. 10^{-4}), qui peut ne pas être optimal pour tous les scénarios. Un seuil dynamique basé sur la vitesse ou le spin de la balle améliorerait la robustesse.
— Les fonctions d'événements ne tiennent pas compte du **mouvement latéral** (v_y) pendant le roulement, ce qui peut entraîner des imprécisions quand la balle a un spin latéral significatif.

2.8.4 Intégration numérique

Le système d'EDO est intégré numériquement avec la fonction `ode` de Scilab, qui supporte diverses méthodes comme **Runge-Kutta ordre 4 (RK4)**. L'intégration est réalisée avec un pas de temps fixe Δt, fixé à 0,1 s dans l'implémentation actuelle.

```
[Pos, rd] = ode('roots', Init0, T0, 0:0.1:Tfin, flightg, 1, events);
```

Ici :
— `Init0` est le vecteur d'état initial.
— `T0` est le temps initial.
— `0:0.1:Tfin` définit la grille de temps avec un pas de 0,1 s.
— `flightg` est la fonction définissant les EDO de la phase de vol.
— `events` est la fonction de détection des impacts.

Points forts :
— L'utilisation d'une **méthode numérique standard** (RK4) garantit stabilité et précision pour la plupart des scénarios.
— La conception modulaire permet de remplacer facilement le solveur (ex. passage à une méthode adaptative comme RK).

Limitations :
— Le **pas de temps fixe** ($\Delta t = 0{,}1$ s) peut être trop grand pour capturer les dynamiques rapides, comme la phase initiale du vol ou le rebond. Un pas plus petit (ex. 0,01 s) ou une méthode adaptative améliorerait la précision.
— Le solveur ne tient pas compte de la **raideur** du système d'EDO, qui peut apparaître dans certains cas limites (ex. taux de spin très élevés). Des méthodes implicites ou semi-implicites (ex. BDF) seraient plus adaptées à ces scénarios.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.8.5 Limitations et évolutions potentielles

Si l'implémentation actuelle est robuste et validée expérimentalement, plusieurs limitations et pistes d'amélioration ont été identifiées :

- **Décroissance du spin** : le modèle actuel suppose que le taux de spin reste constant pendant le vol ($\frac{d\omega}{dt} = 0$). En réalité, le spin décroît à cause de la résistance de l'air, ce qui affecte la force de Magnus et donc la trajectoire. Ajouter des termes de décroissance du spin aux EDO améliorerait la précision, particulièrement pour les longs coups (ex. driver).
  - **Évolution proposée** : introduire un modèle de décroissance du spin basé sur le couple dû à la résistance de l'air :
$$\frac{d\boldsymbol{\omega}}{dt} = -\frac{k\rho A r \boldsymbol{\omega} V}{I}$$
où k est un coefficient empirique, I le moment d'inertie de la balle et V sa vitesse.
- **Coefficients de traînée et de Magnus dynamiques** : le coefficient de traînée C_d et le coefficient de Magnus C_m sont actuellement modélisés par des formules simplifiées. Si ce sont des approximations raisonnables, ils ne capturent pas toute la complexité de l'aérodynamique d'une balle de golf, notamment l'effet des alvéoles.
  - **Évolution proposée** : utiliser des données expérimentales ou des simulations CFD pour obtenir des coefficients C_d et C_m plus précis, en fonction de Re et du taux de spin.
- **Event Detection Robustness** : La détection d'événements actuelle s'appuie sur des seuils fixes, qui peuvent ne pas être optimaux pour tous les scénarios. Par exemple, le seuil de détection de l'impact au sol (10^{-4}) peut être trop grand pour des simulations haute précision.
  - **Évolution proposée** : mettre en place un seuil dynamique ou une interpolation pour localiser précisément l'instant de l'événement entre deux pas de temps.
- **Adaptive Time Stepping** : Le pas de temps fixe ($\Delta t = 0{,}1\,\mathrm{s}$) peut ne pas être efficace pour toutes les phases de la simulation. Par exemple, un pas plus petit est nécessaire pendant la phase de rebond, tandis qu'un pas plus grand peut suffire pendant le roulement.
  - **Évolution proposée** : remplacer la méthode RK4 à pas fixe par une méthode adaptative (ex. RK45 ou Dormand-Prince) qui ajuste le pas de temps selon l'erreur de troncature locale.
- **Mouvement latéral dans les rebonds** : Le modèle de rebond actuel ne tient pas compte du mouvement latéral (v_y) ni du spin latéral (ω_z) pendant le rebond. Cela limite la capacité du modèle à simuler précisément les slices et les hooks.
  - **Évolution proposée** : étendre le modèle de rebond pour inclure les composantes latérales, via un modèle de restitution 3D tenant compte des vitesses verticale et horizontale, ainsi que du spin.
- **Terrain-Specific Parameters** : Les coefficients de frottement (μ) et la résistance au roulement (ρ_g) sont actuellement constants. En réalité, ces paramètres dépendent du type de terrain (ex. green, fairway, rough).
  - **Évolution proposée** : introduire des paramètres selon le terrain et permettre à l'utilisateur de choisir le type de terrain pour chaque simulation.

Ces évolutions amélioreraient la précision et la robustesse du modèle, particulièrement pour les cas limites et les applications haute précision. Toutefois, elles devraient être implémentées incrémentalement et validées sur des données expérimentales pour s'assurer que les améliorations n'introduisent pas de nouvelles erreurs.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 3

Rebonds et roulement

3.1 Rebonds et roulement

La dynamique d'une balle de golf après l'impact au sol est régie par des interactions complexes entre sa vitesse, son spin et les propriétés de la surface. Cette section détaille la modélisation des phases de **rebond** et de **roulement**, d'après les travaux de Woo-Jin Roh et Chong-Won Lee [**?**] et la validation expérimentale de l'implémentation Scilab. La transition du vol vers le rebond puis le roulement est critique pour prédire correctement la position finale de la balle, particulièrement pour les coups courts où ces phases dominent la distance totale.

3.1.1 Caractéristiques de la balle pendant les rebonds

La phase de rebond commence quand la balle touche le sol, détectée par la fonction d'événement décrite en section 2.8.3. Le modèle de rebond repose sur les **coefficients de restitution** et les **forces de frottement** agissant pendant la courte durée de l'impact. Les paramètres et équations clés sont tirés de [**?**], avec des adaptations aux données expérimentales.

3.1.1.1 Angle d'impact et restitution

L'angle d'impact θ_{impact} est défini comme l'angle entre le vecteur vitesse de la balle et le sol au moment de l'impact. Dans l'implémentation, il est calculé par :

$$\theta_{\text{impact}} = \arctan\left(\frac{|v_{ix}|}{|v_{iy}|}\right),$$

où :

- $v_{ix} = \sqrt{v_x^2 + v_z^2}$ est la composante horizontale de la vitesse (dans le plan du sol).
- v_{iy} est la composante verticale de la vitesse.

Le **coefficient de restitution** e_{ball} détermine le rapport entre la vitesse relative après l'impact et celle avant l'impact. Dans le modèle, e_{ball} est défini empiriquement comme une fonction de la composante verticale de vitesse v'_{iy} :

$$e_{\text{ball}} = \begin{cases} 0.510 - 0.0375 \cdot |v'_{iy}| + 0.000903 \cdot |v'_{iy}|^2 & \text{si } |v'_{iy}| < 20\,\text{m/s}, \\ 0.120 & \text{otherwise.} \end{cases}$$

Cette fonction par morceaux rend compte du **comportement non linéaire** de la restitution de la balle, où des vitesses d'impact plus élevées donnent des coefficients de restitution plus faibles à cause des pertes d'énergie par déformation et autres effets dissipatifs.

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

Critique :
- Les coefficients empiriques (0,510, -0,0375, 0,000903) sont spécifiques aux conditions de **green** testées dans [**?**]. Ils peuvent ne pas être universellement applicables à d'autres surfaces (ex. fairway, rough, sable).
- Le seuil de 20 m/s est arbitraire et peut ne pas être valable pour tous les modèles de balles ou types de surface. Une validation expérimentale pour différentes conditions est recommandée.

3.1.1.2 Angle de rebond et mise à jour des vitesses

L'angle de la trajectoire de la balle après le rebond, θ_{rebond}, est calculé par une relation empirique tirée de [**?**] :

$$\theta_{\text{rebond}} = 15.4 \cdot \frac{|\vec{v}_i|}{18.6} \cdot \frac{\theta_{\text{impact}}}{44.4},$$

où $|\vec{v}_i|$ est la norme de la vitesse de la balle à l'impact. Cette équation suppose que l'angle de rebond est proportionnel à la fois à l'angle d'impact et à la norme de la vitesse, mis à l'échelle par des constantes empiriques.

Les vitesses après l'impact sont ensuite calculées par :

$$v'_{ix} = |\vec{v}_i|\cos(\theta_{\text{rebond}}) - |v_{iy}|\sin(\theta_{\text{rebond}}),$$
$$v'_{iy} = |\vec{v}_i|\sin(\theta_{\text{rebond}}) + |v_{iy}|\cos(\theta_{\text{rebond}}).$$

Critique :
- Les constantes empiriques (15,4, 18,6, 44,4) proviennent d'expériences sur des types de gazon spécifiques et peuvent mal se généraliser à d'autres surfaces. Par exemple, une surface plus dure (ex. fairway) donnerait probablement un θ_{rebond} plus faible pour les mêmes conditions d'impact.
- Le modèle suppose un **rebond 2D** (en négligeant le mouvement latéral v_y). Cette simplification peut entraîner des imprécisions pour les coups avec un spin latéral ou une vitesse latérale significatifs (ex. slices ou hooks).

Improvements proposées :
- **Paramètres dépendant du terrain** : introduire un facteur d'échelle dépendant du terrain pour θ_{rebond}. Par exemple :
$$\theta_{\text{rebond}} = k_{\text{terrain}} \cdot 15.4 \cdot \frac{|\vec{v}_i|}{18.6} \cdot \frac{\theta_{\text{impact}}}{44.4},$$
où k_{terrain} vaut 1,0 pour le green, 0,8 pour le fairway et 0,6 pour le rough.
- **Modèle de rebond 3D** : étendre le modèle au mouvement latéral et au spin latéral. Par exemple, la vitesse latérale après rebond v'_{iy} pourrait être mise à jour par :
$$v'_{iy} = e_{\text{lat}} \cdot v_{iy} \cdot \text{sign}(v_y),$$
où e_{lat} est un coefficient de restitution latéral (typiquement 0,8–0,9 pour le green).

3.1.1.3 Frottement et spin pendant le rebond

Pendant le rebond, le frottement entre la balle et le sol affecte à la fois la **vitesse linéaire** et le **spin** de la balle. Le modèle utilise une approche de **frottement de Coulomb**, où la force de frottement est proportionnelle à la force normale et au coefficient de frottement μ. Le coefficient de frottement critique μ_{critic} détermine si la balle **glisse** ou **roule sans glisser** après le rebond :

$$\mu_{\text{critic}} = \frac{2\left(|v'_{ix}| + r|\omega_i|\right)}{7(1+e_{\text{ball}})|v'_{iy}|},$$

où :

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

— r est le rayon de la balle.
— ω_i est le taux de spin initial (backspin ou spin latéral).

Si $\mu < \mu_{\text{critic}}$, la balle glisse, et les vitesses et le spin après l'impact sont mis à jour par :

$$\begin{aligned} v'_{rx} &= v'_{ix} - \mu|v'_{iy}|(1 + e_{\text{ball}}), \\ v'_{ry} &= e_{\text{ball}}|v'_{iy}|, \\ \omega_r &= \omega_i - 5\mu|v'_{iy}|(1 + e_{\text{ball}}). \end{aligned}$$

Si $\mu \geq \mu_{\text{critic}}$, la balle roule sans glisser, et les vitesses et le spin sont mis à jour par :

$$\begin{aligned} v'_{rx} &= \frac{5}{7}v'_{ix} - \frac{2}{7}r\omega_i, \\ v'_{ry} &= e_{\text{ball}}|v'_{iy}|, \\ \omega_r &= -\frac{v'_{rx}}{r}. \end{aligned}$$

Critique :
— Le coefficient de frottement $\mu = 0{,}40$ est une **valeur fixe** pour toutes les surfaces. En réalité, μ varie fortement selon le type de terrain (ex. 0,2–0,3 pour le fairway, 0,5–0,7 pour le rough).
— Le modèle suppose que le **spin latéral** (ω_z) n'affecte pas la dynamique du rebond, ce qui est une simplification. En réalité, le spin latéral peut induire des forces latérales pendant le rebond, particulièrement sur des surfaces irrégulières.

Proposed Improvements :
— **Coefficients de frottement dynamiques** : utiliser des valeurs de μ et μ_{critic} dépendant du terrain. Par exemple :

$$\mu = \begin{cases} 0.40 & \text{pour le green,} \\ 0.20 & \text{pour le fairway,} \\ 0.60 & \text{pour le rough.} \end{cases}$$

— **Spin latéral dans le rebond** : étendre le modèle pour inclure l'effet du spin latéral sur le mouvement latéral. Par exemple, la vitesse latérale après rebond pourrait être mise à jour par :

$$v'_{ry} = e_{\text{lat}} \cdot v_{iy} + k_{\text{spin}} \cdot r\omega_z,$$

où k_{spin} est un coefficient empirique (ex. 0,1–0,2).

3.1.2 Caractéristiques de la balle pendant le roulement

Quand la vitesse verticale de la balle devient négligeable et qu'elle reste en contact avec le sol, la phase de roulement commence. Pendant cette phase, le mouvement de la balle est dominé par le **frottement** et la **résistance au roulement**. Le modèle de roulement repose sur les travaux de Penner [**?**, **?**], qui suppose une relation linéaire entre la décélération et la vitesse.

3.1.2.1 Résistance au roulement

La force de résistance au roulement est modélisée comme un terme de **frottement visqueux**, proportionnel à la vitesse de la balle :

$$\mathbf{F}_{\text{roll}} = -\frac{5}{7}\rho_g g\mathbf{v},$$

où :
— $\rho_g = 0{,}131$ est le coefficient de résistance au roulement pour un green.
— g est l'accélération de la pesanteur.
— $\mathbf{v} = (v_x, v_y, v_z)$ est le vecteur vitesse.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Les équations du mouvement pour la phase de roulement sont :

$$\frac{d}{dt}\begin{bmatrix} x \\ v_x \\ y \\ v_y \\ z \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ -\frac{5}{7}\rho_g g \frac{v_x}{\|\mathbf{v}\|} \\ v_y \\ -\frac{5}{7}\rho_g g \frac{v_y}{\|\mathbf{v}\|} \\ v_z \\ -\frac{5}{7}\rho_g g \frac{v_z}{\|\mathbf{v}\|} \end{bmatrix}.$$

Critique :

- Le coefficient de résistance au roulement $\rho_g = 0{,}131$ est **fixe** et spécifique au green. Pour d'autres surfaces, ρ_g peut varier significativement (ex. 0,065–0,196 pour le fairway [**?**]).
- Le modèle suppose que la balle **glisse** jusqu'à l'arrêt, ce qui n'est pas physiquement exact. En réalité, la balle passe en **roulement sans glissement** quand la force de frottement suffit à empêcher le glissement.

Proposed Improvements :

- **Résistance au roulement selon le terrain** : utiliser des valeurs de ρ_g dépendant du terrain. Par exemple :

$$\rho_g = \begin{cases} 0.131 & \text{pour le green,} \\ 0.10 & \text{pour le fairway,} \\ 0.30 & \text{pour le rough.} \end{cases}$$

- **Roulement sans glissement** : ajouter une condition de transition vers le roulement sans glissement quand la force de frottement équilibre la résistance au roulement. Par exemple :

$$\text{Si } |\mathbf{v}| < v_{\text{roll}}, \text{ alors } \frac{d\mathbf{v}}{dt} = 0,$$

où v_{roll} est le seuil de vitesse pour le roulement sans glissement (ex. 0,1 m/s).

3.1.2.2 Condition d'arrêt

La phase de roulement se termine quand la vitesse horizontale de la balle passe sous un seuil, détecté par la fonction `events2` :

$$\text{value} = v_x - 0.$$

Cette condition termine la simulation quand $v_x \leq 0$.

Critique :

- La condition d'arrêt ne vérifie que la **vitesse horizontale** v_x et ignore v_y et v_z. Cela peut provoquer un arrêt prématuré de la simulation si la balle a une composante de vitesse latérale ou verticale significative.
- Le seuil de 0 est arbitraire et peut ne pas rendre compte du mouvement de la balle pour tous les scénarios.

Proposed Improvements :

- **Condition d'arrêt 3D** : utiliser la norme du vecteur vitesse comme critère d'arrêt :

$$\text{value} = \|\mathbf{v}\| - v_{\text{stop}},$$

où $v_{\text{stop}} = 10^{-4}$ m/s est un petit seuil.

- **Seuil dynamique** : utiliser un seuil dynamique basé sur le spin de la balle ou la rugosité de la surface. Par exemple :

$$v_{\text{stop}} = k_{\text{stop}} \cdot r\omega,$$

où k_{stop} est un coefficient empirique (ex. 0,01).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.1.3 Limitations et évolutions potentielles

Si le modèle de rebonds et de roulement repose sur des principes physiques bien établis et a été validé expérimentalement, plusieurs limitations et pistes d'amélioration ont été identifiées :

- **Coefficients fixes** : le coefficient de restitution e_{ball}, le coefficient de frottement μ et la résistance au roulement ρ_g sont actuellement fixés. Ces coefficients dépendent du **type de terrain**, du **modèle de balle** et des **conditions d'impact**.
  - **Évolution proposée** : introduire un **jeu de paramètres selon le terrain** et permettre à l'utilisateur de choisir le type de terrain pour chaque simulation. Par exemple, créer une table de correspondance pour e_{ball}, μ et ρ_g selon la surface (green, fairway, rough, sable).
- **Modèle de rebond 2D** : le modèle actuel suppose un mouvement 2D (en négligeant les composantes latérales v_y et ω_z). Cela limite la capacité du modèle à simuler précisément les **slices** et les **hooks**.
  - **Évolution proposée** : étendre le modèle de rebond en 3D en incluant le mouvement latéral et le spin latéral. Par exemple, utiliser un modèle de restitution 3D qui tient compte de toutes les composantes de vitesse et de spin :

$$
\begin{aligned}
v'_{ix} &= e_{\text{long}} v_{ix} - \mu(1 + e_{\text{long}})|v_{iy}|, \\
v'_{iy} &= e_{\text{vert}} v_{iy}, \\
v'_{iz} &= e_{\text{lat}} v_{iz} + k_{\text{spin}} r \omega_z, \\
\omega'_x &= \omega_x, \\
\omega'_y &= \omega_y - \frac{5}{2}\mu(1 + e_{\text{vert}})|v_{iy}|, \\
\omega'_z &= \omega_z - \frac{5}{2}\mu(1 + e_{\text{lat}})|v_{iz}|,
\end{aligned}
$$

    où e_{long}, e_{vert} et e_{lat} sont les coefficients de restitution longitudinal, vertical et latéral, respectivement.
- **Décroissance du spin pendant le roulement** : le modèle actuel suppose que le spin reste constant pendant le roulement. En réalité, le spin décroît à cause de la **résistance au roulement** et du **frottement**.
  - **Évolution proposée** : ajouter un terme de décroissance du spin aux équations de roulement. Par exemple :

$$
\frac{d\omega}{dt} = -\frac{k \rho_g g r \omega}{I},
$$

    où k est un coefficient empirique (ex. 0,1) et I le moment d'inertie de la balle.
- **Rugosité et pente de la surface** : le modèle actuel suppose une surface **plane et uniforme**. En réalité, la surface peut avoir une **pente** ou une **rugosité**, qui peuvent affecter le mouvement de la balle.
  - **Évolution proposée** : incorporer un **modèle de pente** dans les équations de roulement. Par exemple, si la surface a une pente α selon x, la composante de la force de gravité le long de la pente est :

$$
F_{g,x} = mg \sin(\alpha).
$$

    Cette force peut être ajoutée au terme de résistance au roulement.
- **Validation pour différentes balles** : le modèle actuel utilise des coefficients empiriques dérivés de modèles de balles spécifiques (ex. Mizuno JPX, Decathlon). Ces coefficients peuvent ne pas s'appliquer à toutes les balles de golf.
  - **Évolution proposée** : mener des expériences pour obtenir des **coefficients spécifiques à chaque balle** pour e_{ball}, μ et ρ_g. Stocker ces coefficients dans une table indexée par modèle de balle.

Ces évolutions amélioreraient significativement la précision et la robustesse du modèle de rebonds et roulement, particulièrement pour les cas limites comme les coups à fort spin, les surfaces irrégulières ou les modèles de balles non standards. Toutefois, chaque amélioration devrait être **validée sur des données expérimentales** pour s'assurer qu'elle n'introduit pas de nouvelles erreurs ou incohérences.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2 Explication du code Scilab de simulation de la balle de golf

Cette section donne une explication détaillée du code Scilab utilisé pour simuler le vol, les rebonds et le roulement d'une balle de golf. Le code comporte des simplifications pour garantir un calcul rapide, car il est destiné à traiter en temps réel les données radar du début de vol de la balle (discuté au chapitre suivant).

3.2.1 La fonction `flightg`

La fonction `flightg` est le cœur de la simulation de la phase de vol. Elle calcule les dérivées temporelles des variables d'état (position, vitesse, spin) à partir des forces physiques agissant sur la balle : traînée, portance de Magnus et gravité.

3.2.1.1 Entrées et sorties

- **Entrées** :
  - `t` : temps courant (s).
  - `v` : vecteur d'état de taille 12 contenant :
    - `v(1)` : position x (m).
    - `v(2)` : vitesse v_x (m/s).
    - `v(3)` : position y (m).
    - `v(4)` : vitesse v_y (m/s).
    - `v(5)` : position z (m).
    - `v(6)` : vitesse v_z (m/s).
    - `v(7)` : spin ω_x (rad/s).
    - `v(8)` : spin ω_y (rad/s).
    - `v(9)` : spin ω_z (rad/s).
    - `v(10)` : coefficient de Magnus C_m.
    - `v(11)` : masse m (kg).
    - `v(12)` : diamètre D (m).
- **Sortie** :
  - `vprime` : vecteur des dérivées temporelles des variables d'état.

3.2.1.2 Logique interne

La fonction `flightg` suit ces étapes pour calculer les dérivées :

1. **Initialisation des constantes physiques** :
   - La masse m et le diamètre D sont extraits du vecteur d'état.
   - La masse volumique de l'air ρ_{air} est calculée pour une température fixée $T = 20°C$:
   $$\rho_{\text{air}} = 1.292 \times \frac{273.15}{273.15 + T}$$
   - La section droite A de la balle est calculée :
   $$A = \pi \left(\frac{D}{2}\right)^2$$
2. **Calcul de la vitesse et du spin** :
   - La norme de la vitesse V_m est calculée :
   $$V_m = \sqrt{v_x^2 + v_y^2 + v_z^2}$$
   - La norme du spin W_m est calculée :
   $$W_m = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3. **Calcul du coefficient de traînée** :
   — Le nombre de Reynolds Re est calculé :
   $$\text{Re} = \frac{V_m \times D}{1.5 \times 10^{-5}}$$
   — Le coefficient de traînée C_d est calculé avec la formule de White :
   $$C_d = 0.36 + \frac{24}{\text{Re}} + \frac{6}{1 + \sqrt{\text{Re}}}$$
4. **Calcul de la force de traînée** :
   — La norme de la force de traînée D est calculée :
   $$D = \frac{1}{2}\rho_{\text{air}} C_d A V_m^2$$
   — Les composantes de la force de traînée sont calculées selon la direction de la vitesse :
   $$F_{dx} = -\frac{D \times v_x}{V_m}, \quad F_{dy} = -\frac{D \times v_y}{V_m}, \quad F_{dz} = -\frac{D \times v_z}{V_m}$$
5. **Calcul de la force de Magnus** :
   — La norme de la force de Magnus M est calculée avec le coefficient empirique C_{l1} stocké dans v(10) :
   $$M = \frac{1}{2}\rho_{\text{air}} A \left(\frac{D}{2}\right) C_{l1} V_m$$
   — Les composantes de la force de Magnus sont calculées par le produit vectoriel $\boldsymbol{\omega} \times \mathbf{v}$:
   $$F_{mx} = M \times (\omega_y v_z - \omega_z v_y), \quad F_{my} = M \times (\omega_z v_x - \omega_x v_z), \quad F_{mz} = M \times (\omega_x v_y - \omega_y v_x)$$
6. **Calcul des dérivées temporelles** :
   — Les dérivées des positions sont les vitesses :
   $$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y, \quad \frac{dz}{dt} = v_z$$
   — Les dérivées des vitesses sont calculées à partir des forces de traînée, de Magnus et de gravité :
   $$\frac{dv_x}{dt} = \frac{F_{dx} + F_{mx}}{m}, \quad \frac{dv_y}{dt} = -g + \frac{F_{dy} + F_{my}}{m}, \quad \frac{dv_z}{dt} = \frac{F_{dz} + F_{mz}}{m}$$
   — Les dérivées des composantes de spin sont supposées nulles (pas de décroissance du spin en vol, par simplification) :
   $$\frac{d\omega_x}{dt} = \frac{d\omega_y}{dt} = \frac{d\omega_z}{dt} = 0$$

Note : la simplification consistant à supposer aucune décroissance du spin en vol est faite pour garantir un calcul rapide, le modèle étant destiné à traiter en temps réel les données radar du début de vol de la balle (discuté au chapitre suivant).

3.2.2 La fonction `rolling`

La fonction `rolling` modélise la phase de roulement de la balle après son atterrissage. Elle calcule les dérivées temporelles des variables d'état, en tenant compte du frottement et de la résistance au roulement.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2.2.1 Entrées et sorties

— **Inputs** :
  — `t` : Current time (s).
  — `v` : vecteur d'état de taille 10 contenant :
    — `v(1)` : Position x (m).
    — `v(2)` : Velocity v_x (m/s).
    — `v(3)` : Position y (m).
    — `v(4)` : Velocity v_y (m/s).
    — `v(5)` : Position z (m).
    — `v(6)` : Velocity v_z (m/s).
    — `v(7)` : Spin ω_x (rad/s).
    — `v(8)` : Spin ω_y (rad/s).
    — `v(9)` : Spin ω_z (rad/s).
    — `v(10)` : coefficient de résistance au roulement.
— **Sortie** :
  — `vprime` : Vector of time derivatives of the state variables.

3.2.2.2 Internal Logic

La fonction `rolling` suit ces étapes :

1. **Physical Constants Initialization** :
   — La masse m est fixée à $0{,}04545\,\text{kg}$.
   — La masse volumique de l'air ρ_{air} est calculée pour une température de $20°C$.
   — Le coefficient de résistance au roulement ρ_g est fixé à $0{,}131$ (valeur typique d'un green).
   — L'accélération de la pesanteur g est fixée à $9{,}81\,\text{m/s}^2$.
2. **Calcul des dérivées temporelles** :
   — Les dérivées des positions sont les vitesses :

$$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y, \quad \frac{dz}{dt} = v_z$$

   — Les dérivées des vitesses sont calculées avec un modèle de frottement proportionnel à la vitesse :

$$\frac{dv_x}{dt} = -\frac{5}{7}\rho_g g \frac{v_x}{\sqrt{v_x^2 + v_y^2 + v_z^2}}, \quad \frac{dv_z}{dt} = -\frac{5}{7}\rho_g g \frac{v_z}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

   — La dérivée de v_y tient compte de la pente du terrain via la fonction `dAltiSol` :

$$\frac{dv_y}{dt} = -\frac{5}{7}\rho_g g \frac{\text{dAltiSol}(v_x, v_z)}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

   — Les dérivées des composantes de spin sont supposées nulles (pas de décroissance du spin en roulement, par simplification).

Note : les simplifications du modèle de roulement visent à garantir un calcul rapide, le modèle étant destiné à traiter en temps réel les données radar du début de vol de la balle (discuté au chapitre suivant).

3.2.3 Les fonctions `AltiSol` et `dAltiSol`

Les fonctions `AltiSol` et `dAltiSol` servent à modéliser l'altitude du terrain et sa pente, respectivement.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2.3.1 La fonction `AltiSol`

- **Rôle** : calcule l'altitude du terrain à la position (x, z).
- **Inputs** :
  - `Longueur` : position x (m).
  - `largeur` : position z (m).
- **Output** :
  - `Altitude` : altitude du terrain à la position (x, z) (m).
- **Logique** :
  - Si $x < 30$ m, l'altitude est mise à 0 (terrain plat).
  - Sinon, l'altitude est aussi mise à 0, mais une ligne commentée suggère un modèle potentiel pour un terrain en pente.

3.2.3.2 La fonction `dAltiSol`

- **Rôle** : calcule la dérivée de l'altitude du terrain par rapport à la position horizontale, utilisée pour modéliser la pente pendant le roulement.
- **Inputs** :
  - `vLongueur` : vitesse v_x (m/s).
  - `vlargeur` : vitesse v_z (m/s).
- **Output** :
  - `vAltitude` : dérivée de l'altitude du terrain.
- **Logic** :
  - Si $v_x < 30$ m/s, la dérivée est mise à 0 (terrain plat).
  - Sinon, la dérivée est aussi mise à 0, mais une ligne commentée suggère un modèle potentiel pour un terrain en pente.

3.2.4 Fonctions de détection d'événements

Les fonctions `events` et `events2` servent à détecter les événements discrets pendant la simulation, comme l'impact de la balle au sol ou sa condition d'arrêt.

3.2.4.1 La fonction `events`

- **Rôle** : détecte quand la balle touche le sol ($y \leq \text{Altitude}(x, z)$).
- **Inputs** :
  - `t` : Current time (s).
  - `x` : vecteur d'état.
- **Output** :
  - `value` : valeur utilisée par le solveur d'EDO pour détecter l'événement.
- **Logic** :
  - Si la hauteur `x(3)` de la balle est inférieure à l'altitude du terrain à la position `(x(1), x(5))`, l'événement est déclenché (`value = 0`).
  - Sinon, la fonction retourne la différence entre la hauteur de la balle et l'altitude du terrain.

3.2.4.2 La fonction `events2`

- **Rôle** : détecte quand la balle s'arrête de rouler ($v_x \leq 0$).
- **Inputs** :
  - `t` : Current time (s).
  - `x` : State vector.
- **Output** :
  - `value` : Value used by the ODE solver to detect the event.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

— **Logic** :
  — L'événement est déclenché quand la vitesse horizontale `x(2)` est inférieure ou égale à 0.

3.2.5 Intégration numérique avec `ode`

La solution numérique des équations différentielles est réalisée avec la fonction `ode` de Scilab. Cette fonction utilise par défaut la méthode de Runge-Kutta ordre 4 (RK4) pour intégrer le système d'EDO.

3.2.5.1 Appel de la fonction `ode`

Dans la fonction principale `Golfball`, la fonction `ode` est appelée comme suit :

```
[Pos, rd] = ode('roots', Init0, T0, 0:0.1:Tfin, flightg, 1, events);
```

Listing 3.1 – Appel de la fonction `ode` pour la phase de vol

— **Paramètres** :
  — `'roots'` : méthode d'intégration (RK4 par défaut).
  — `Init0` : vecteur d'état initial.
  — `T0` : temps initial (généralement 0).
  — `0:0.1:Tfin` : grille de temps avec un pas de 0,1 s.
  — `flightg` : fonction définissant les EDO de la phase de vol.
  — `1` : indique que les événements doivent être détectés.
  — `events` : fonction de détection des impacts.
— **Sorties** :
  — `Pos` : matrice contenant le vecteur d'état à chaque pas de temps.
  — `rd` : vecteur contenant les instants auxquels les événements (ex. impacts) se produisent.

Note : le pas de temps fixe de 0,1 s est choisi pour équilibrer efficacité de calcul et précision, le modèle étant destiné à traiter en temps réel les données radar du début de vol de la balle (discuté au chapitre suivant).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 4

Matériel



Figure 4.1 – La valise

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

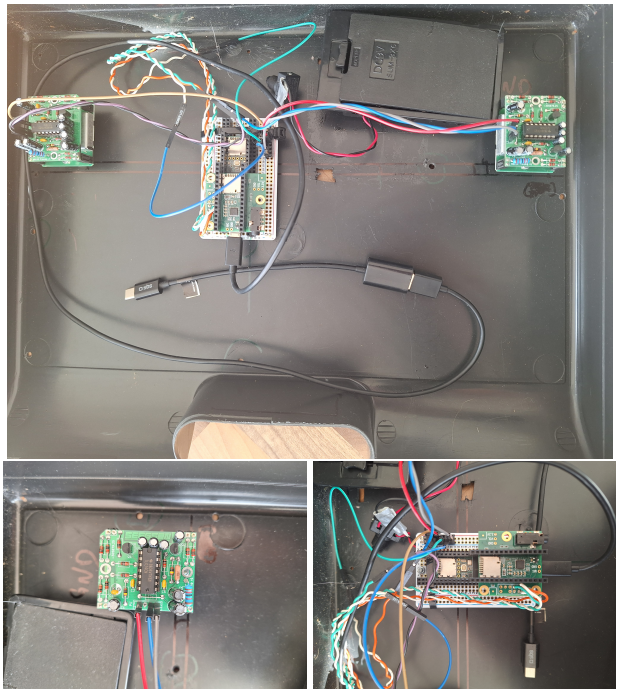


FIGURE 4.2 – Intérieur avec Arduino, radar, amplificateur, cartes audio



FIGURE 4.3 – Tiny Arduino

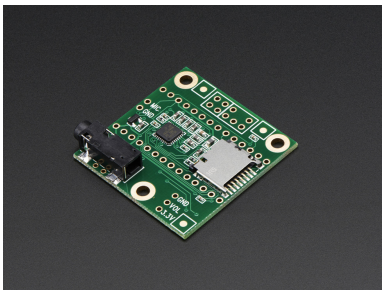


FIGURE 4.4 – Carte audio

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

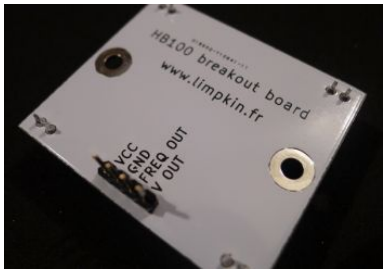


Figure 4.5 – Carte amplificateur audio



Figure 4.6 – Carte radar

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 5

Analyse des données

5.1 Captures radar filtrées

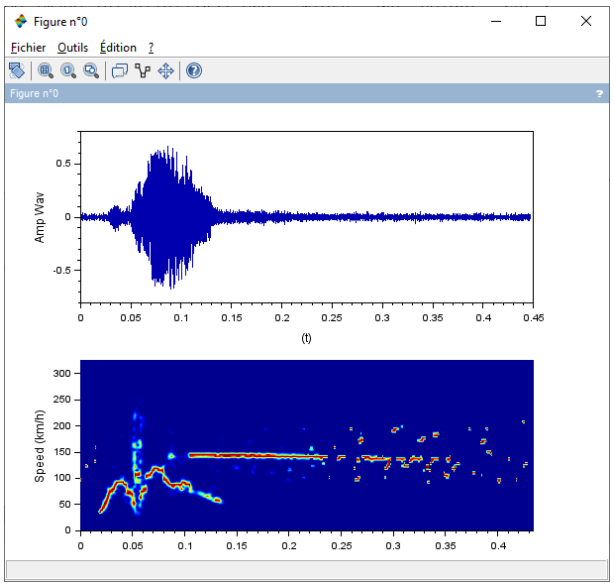


FIGURE 5.1 – Captures WAV radar et leur écho

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

5.1.1 Filtrage

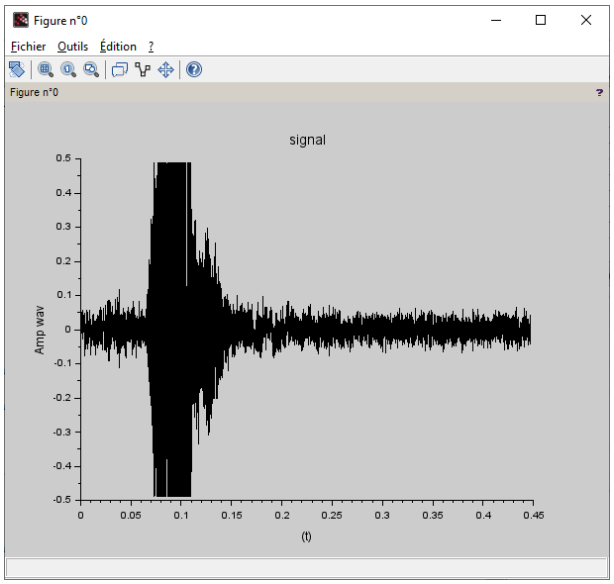


FIGURE 5.2 – WAV brut capturé

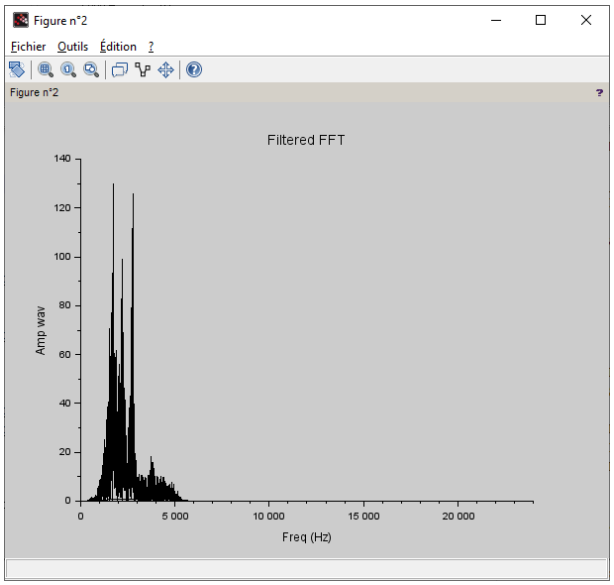


FIGURE 5.3 – FFT du brut filtré

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

5.1.2 Résultat final

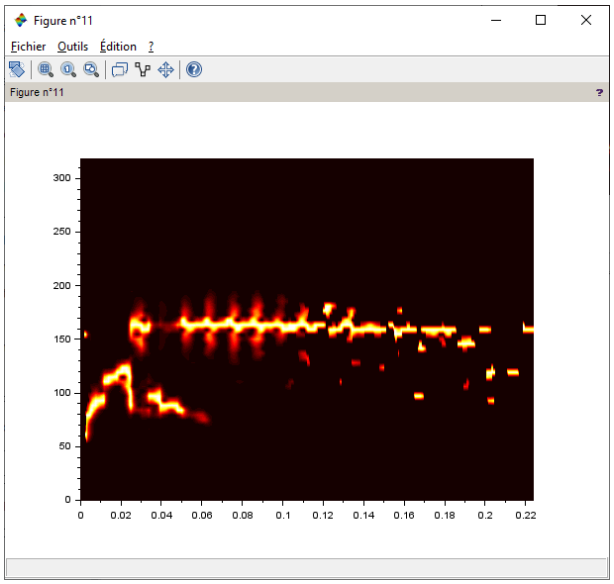


FIGURE 5.4 – Résultats de mon algorithme de densité de puissance pour identifier vitesse et backspin

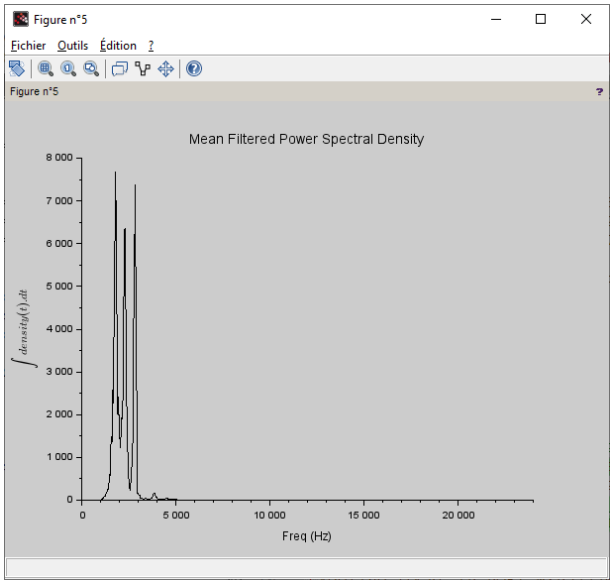


FIGURE 5.5 – Résultats de la densité spectrale de puissance filtrée en moyenne et normalisée

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

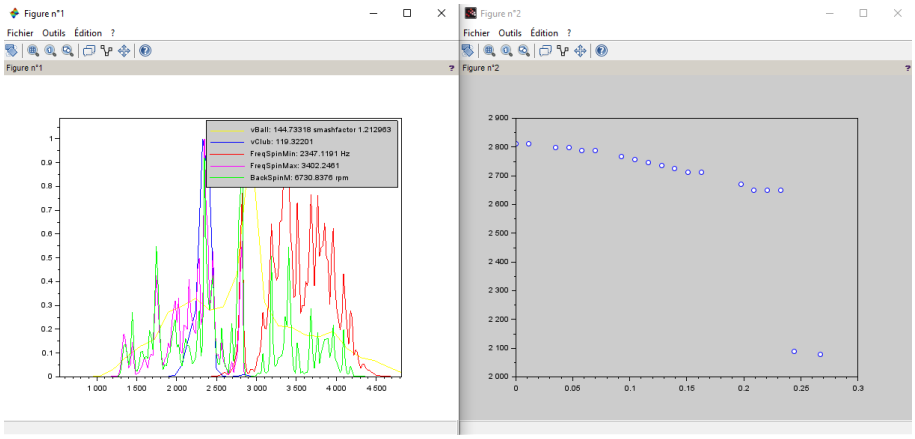


FIGURE 5.6 – Résultats du filtrage de la densité et de l'identification des caractéristiques

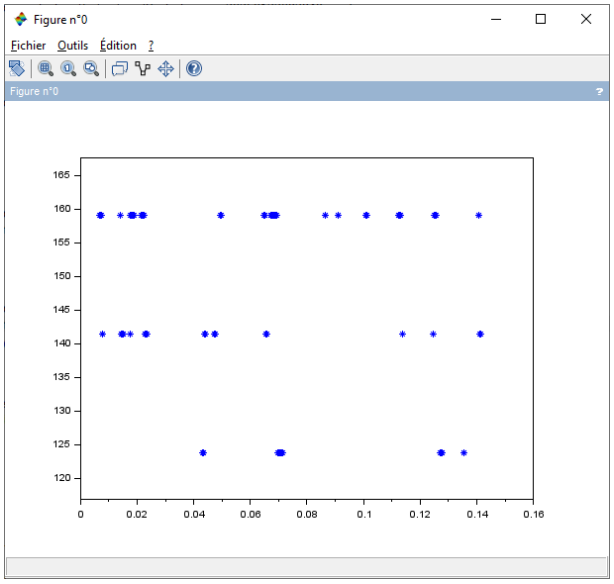


FIGURE 5.7 – Échos de vitesse et de backspin

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 6

Pipe de traitement du signal radar Doppler : de Scilab à Python

Ce chapitre documente la chaîne complète de traitement du signal permettant d'extraire la vitesse de la balle, le spin total et l'azimut de départ à partir des deux radars Doppler 10,525 GHz (acquisition audio à $F_s = 44100$ Hz, un canal par radar), les corrections apportées au code Scilab historique (`sci/FFT_Mat.sce`, commit `73ef2d8`), et le port Python (`sci/pipe_validation.py`) utilisé pour la validation aveugle sur les 9 captures F7 du 2019-02-24 plus 2 signaux modèle générés.

6.1 Géométrie des deux radars et observables Doppler

Chaque radar ne mesure que la projection de la vitesse de la balle sur sa ligne de visée :

$$f_i = \frac{2}{\lambda}\,\mathbf{V}\cdot\mathbf{u}_i = K_V\,V_{r,i}, \qquad K_V \approx 19.49\ \text{Hz/(km/h)}, \quad \lambda = c/f_0 \approx 0.02848\ \text{m} \tag{6.1}$$

Avec les radars en $R_1 = (0, -b/2)$ et $R_2 = (0, +b/2)$ dans le plan horizontal (axe x = direction nominale du tir, z = latéral, $b = 0{,}30$ m), balle en $P = (x, z)$, $L_1 \approx L_2 \approx L$:

$$f_1 = K\,\frac{V_x x + V_z (z + b/2)}{L}, \qquad f_2 = K\,\frac{V_x x + V_z (z - b/2)}{L} \tag{6.2}$$

En sommant, les termes en $\pm b/2$ s'annulent (mode commun) ; en soustrayant, ce sont les termes longitudinaux qui s'annulent (mode différentiel) :

$$f_C = \frac{f_1 + f_2}{2} \approx K\,V_x \quad \text{« à quelle vitesse elle avance »} \tag{6.3}$$

$$f_\Delta = \frac{f_1 - f_2}{2} = K\,\frac{V_z\,b}{2L} \quad \text{« à quelle vitesse elle dérive latéralement, vue à travers la parallaxe } b/L \text{ »} \tag{6.4}$$

Le rapport redonne la pente du vecteur vitesse, c'est-à-dire la tangente de l'angle de départ horizontal α (angle entre $\mathbf{V}$ et l'axe x dans le plan x–z) :

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

$$\tan\alpha = \frac{V_z}{V_x} = \frac{2L}{b}\,\frac{f_\Delta}{f_C} \quad\Longrightarrow\quad \alpha(t) = \arctan\!\left[\frac{2L(t)}{b}\,\frac{f_\Delta(t)}{f_C(t)}\right], \qquad L(t) = L_0 + \int_{t_0}^{t} V_x(\tau)\,d\tau \tag{6.5}$$

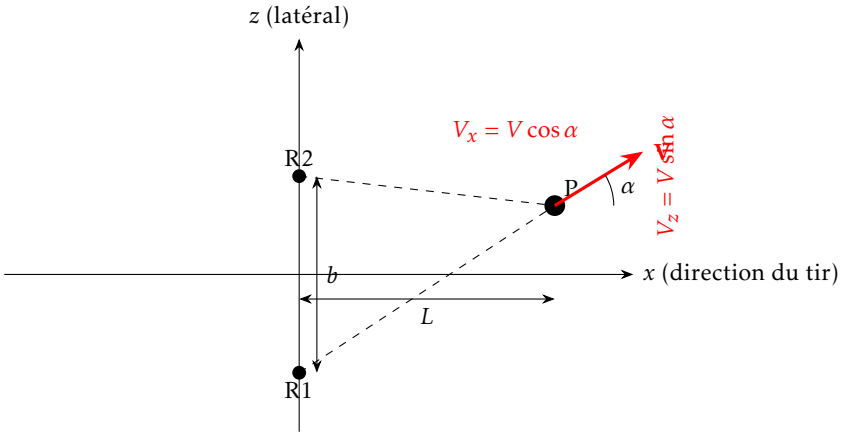


FIGURE 6.1 – Vue de dessus du dispositif à deux radars. f_C porte V_x (vitesse), f_Δ porte V_z (azimut α dans le plan x–z). Le facteur $2L/b$ corrige la parallaxe : plus la balle est loin, plus la différence de fréquence est faible pour le même V_z.

6.2 STFT et suivi du ridge

Pour chaque canal, une transformée de Fourier à court terme donne le spectrogramme $P_i(k,m) = |X_i(k,m)|^2$:

- Scilab `animDensite` (`FFT_Mat.sce`) : fenêtre de Hann 512, pas 16, seuillage en dB par trame, normalisé par le max de la trame ;
- Python `spectrogram` (`pipe_validation.py`) : fenêtre de Hann $N = 2048$ (46 ms), pas 1024 (recouvrement 50%), interpolation parabolique sous-bin du pic.

La fréquence ridge de la trame m est le bin du pic affiné par une parabole :

$$k^* = \arg\max_k P_m(k), \qquad \delta = \frac{0.5\,(P_{k^*-1} - P_{k^*+1})}{P_{k^*-1} - 2P_{k^*} + P_{k^*+1}}, \qquad f_{ridge}(m) = (k^* + \delta)\,\frac{F_s}{N} \tag{6.6}$$

L'instant d'impact est la trame la plus énergétique (le club métallique donne un écho fort, écrêté) :

$$t_0 = \arg\max_m \sum_k P_m(k) \tag{6.7}$$

6.3 Vitesse : médiane du ridge post-impact vs max global

Le max global du spectre sommé dans le temps, $\hat{f}_B = f_5[\arg\max_k \sum_m M_5(m,k)^2]$ (Scilab `Info`), est dominé par le fort écho du club écrêté (6,6–14,8% des échantillons à $\pm 0{,}489$), dont le spectre est large et plat. La balle produit un ridge étroit qui *durée* tout le vol. D'où l'estimateur Python :

$$\hat{V} = \frac{1}{19.49}\,\underset{m\in[t_0,\,t_0+50\,\mathrm{ms}]}{\mathrm{median}}\, f_{ridge}(m) \tag{6.8}$$

Chaque trame vote pour une fréquence, sans pondération : la médiane tolère jusqu'à 50% de trames aberrantes (résidu du club, faux lobes), tandis que la somme pondérée par l'énergie laisse une seule trame fausse mais forte tirer le résultat. Mesuré sur les 11 CSV : MAE 5,7% radiale, 1,9% après correction géométrique $\cos 20°$. Sur microcontrôleur (Arduino), la détection d'impact est en $O(1)$ par trame et un max sur la fenêtre post-impact suffit quand t_0 est fiable ; la médiane est la marge de sécurité quand il ne l'est pas.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

6.4 Spin : périodicité du patch vs excursion spectrale

La balle est partiellement recouverte de ruban aluminium (un diffuseur dominant, « le patch »). Deux estimateurs du spin *total* existent :

1. **Excursion spectrale** (Scilab `GetSwing/Info2`) : la vitesse du point de surface est $v_t = r\omega$, donc l'étalement Doppler est

$$\Delta f_{spin} = \frac{f_+ - f_-}{2} = \frac{2r\omega}{\lambda}, \qquad SpinTotal_{RPM} = \frac{\Delta f_{spin}}{K_S}, \quad K_S \approx 0.1567 \text{ Hz/RPM} \tag{6.9}$$

   Cela utilise les deux *bords extrêmes* du spectre micro-Doppler – les points les plus faibles, donc les plus noyés dans le bruit et l'écrêtage ADC. MAE mesurée : 41,4% (`GetSwing`) à 51–68% (variantes).
2. **Périodicité du patch** (Python) : le patch produit un « flash » radar par tour, quelle que soit l'inclinaison β de l'axe de spin, donc l'enveloppe de Hilbert du signal post-impact contient une raie à

$$f_{rot} = \frac{SpinTotal}{60} \quad \Longrightarrow \quad \boxed{SpinTotal_{period} = 60\, f_{rot}} \tag{6.10}$$

   f_{rot} est le pic du spectre d'enveloppe au meilleur score d'harmoniques $\sum_q P(k_q)$ pour $f_q \approx k f_{rot}$ (évite les sous-harmoniques et le cas dégénéré à 2 flashs par tour quand l'axe de spin est aligné avec la ligne de visée). MAE mesurée : **19,9%** sur les 9 vrais coups, et −0,1% (3996 vs 4000 tr/min) sur le signal modèle propre.

Les deux estimateurs mesurent le spin *total* $\omega = \sqrt{\omega_{back}^2 + \omega_{side}^2}$, pas le backspin : la fréquence du flash vaut $\omega/2\pi$ quel que soit β, et l'excursion vaut $2r\omega/\lambda$. Séparer BackSpin/SideSpin exige β, identifié ailleurs (courbure de $\alpha(t)$, ou le `SpinAxis` de `GetSwing` : $\beta \approx \text{asin}[Spin_3/\sqrt{((Spin_1 + Spin_2)/2)^2 + Spin_3^2}]$ — qualitatif seulement : MAE ≈ 15–18°).

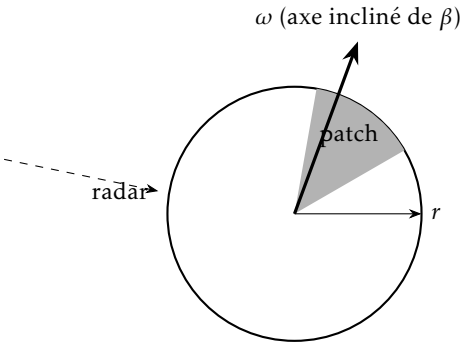


FIGURE 6.2 – Le patch aluminium comme diffuseur dominant unique. La périodicité du flash comme l'excursion spectrale mesurent le spin *total* ω ; aucune des deux ne sépare backspin et sidespin.

6.5 Appariement stéréo des deux ridges

Suivre indépendamment le max FFT de chaque radar peut choisir deux lobes micro-Doppler différents et produire un faux azimut (erreurs > 20° observées). Le tracker Python les apparie : connaissant f_1 (ridge de R1), f_2

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

est cherché dans le spectre de R2 à ±300 Hz de f_1, avec la contrainte de cohérence $|f_1 - f_2| \lesssim 30$ Hz (géométrie, $b = 0{,}30$ m). Puis $\alpha(t)$ découle de l'équation ci-dessus, avec

$$\alpha_0 = \underset{t\in[t_0,t_0+50\,\mathrm{ms}]}{\mathrm{median}}\ \alpha(t), \qquad \Delta\alpha_{BF} = \underset{W_F}{\mathrm{median}}(\alpha) - \underset{W_0}{\mathrm{median}}(\alpha) \tag{6.11}$$

Mesuré : α_0 résolu 9/9 ($-3{,}1°$ à $-0{,}2°$) ; signe de $\Delta\alpha$ vs sidespin de référence 0/9 car les fichiers de référence n'encodent que |sidespin| (le dataset signé gauche/droite est la prochaine priorité).

6.6 Contrôles qualité (« non identifiable » est une sortie valide)

- symétrie des bords micro-Doppler : $|(f_C - f_-) - (f_+ - f_C)| < \eta\,\Delta f_{spin}$, $\eta = 0{,}2$;
- cohérence inter-radars : $|Spin_1 - Spin_2|$/moyenne < 20% ;
- concordance excursion/périodicité < 20% pour fusionner les deux estimateurs de spin, sinon périodicité seule ;
- divergence de vitesse inter-radars > 15% ⇒ rejet du canal au faux lobe ;
- objectifs de validation : $\mathrm{MAE}_V < 2$ km/h, $\mathrm{MAE}_{spin} < 500$ tr/min (8%), $\mathrm{MAE}_{\alpha_0} < 1°$, > 90% de signes de courbure corrects, $\mathrm{MAE}_\beta < 7°$ puis 3–5°.

6.7 Scilab → Python : corrections et résultats de validation

Corrections commitées dans `sci/FFT_Mat.sce` (commit `73ef2d8`, mergées dans `main`) :

- `deltaF` : ajout du facteur 3,6 manquant (km/h → m/s) et harmonisation de $r = 0{,}04267/2$, désormais cohérent avec `Freq2RpmSpin` ;
- `animDensite2/3` : garder la *moitié basse* du spectre ($M1(:, 1 : \$/2)$) correspondant au vecteur fréquence (avant : `$/2:$-1`, moitié haute, mapping cassé) ; seuillage mis à 0 au lieu de −1000 (qui écrasait le pic après normalisation) ;
- `Enveloppe` : démodulation I/Q à la porteuse centrale $f_c = (f_b + f_h)/2$ au lieu de $\cos(t \cdot F_s) = \cos(n)$ (démodulation à $F_s/2\pi \approx 7$ kHz, hors bande).

Validation aveugle sur les 11 fichiers CSV (références encodées dans les noms de fichiers), port des pipes Scilab exacts (README « Simples Scilab Commands » + `testsFFT.sci` ligne 1029) vs le pipe Python :

Table 6.1 – Résultats mesurés, 11 CSV (9 vrais coups F7 + 2 signaux modèle).

Estimateur	MAE V_{balle} radiale	MAE corr. cos 20°	MAE SpinTotal
Scilab `GetSwing` (port)	5,7%	≈1,9%	41,4% (excursion)
Variantes robustes Scilab	5,7%	≈1,9%	60–68%
Python `pipe_validation.py`	**5,7%**	**1,9%**	**19,9%** (périodicité)

Conclusion : la chaîne Scilab est *bonne pour la vitesse et l'impact* (MAE identique au Python) et indicative pour le spin ; le pipe Python ajoute l'estimateur de spin par périodicité du patch, l'appariement stéréo pour α, et les contrôles qualité – les gains viennent du choix du bon observable, pas d'une mathématique plus lourde.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

6.8 Pipe de traitement

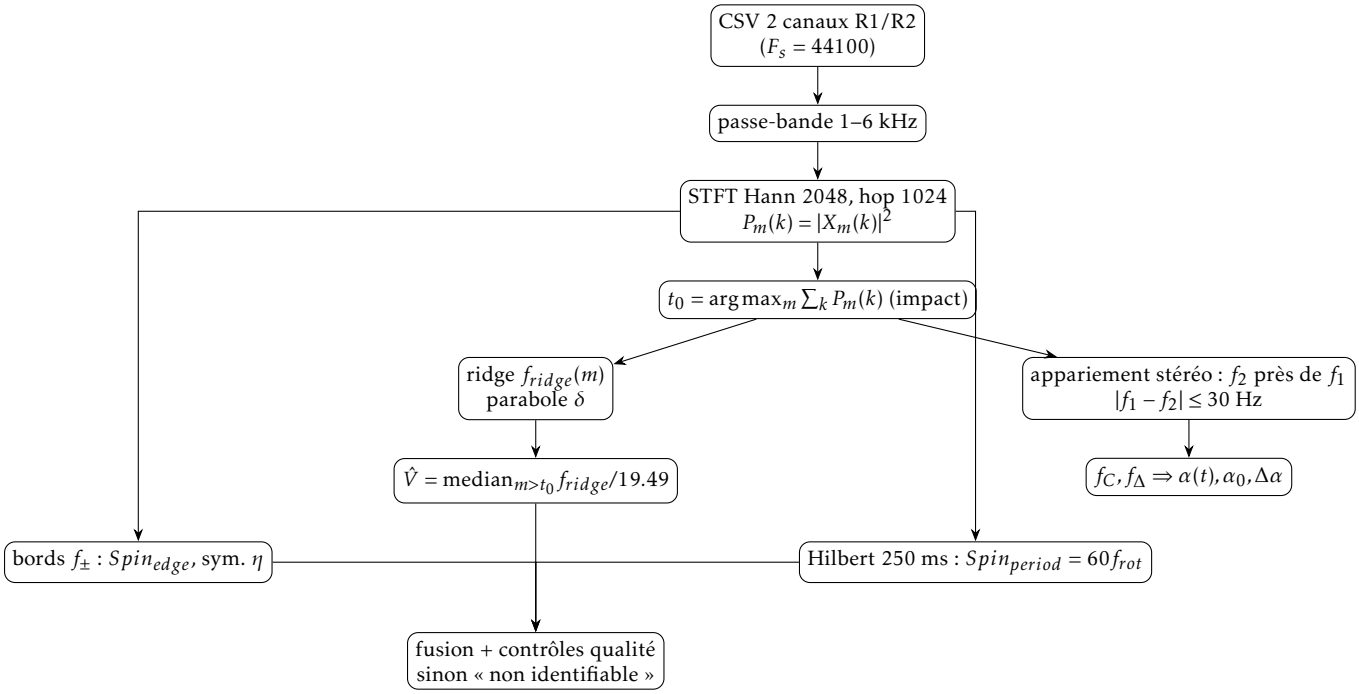


Figure 6.3 – Pipe de validation Python (`sci/pipe_validation.py`). Équivalents Scilab : `animDensite` (STFT), `Info/GetSwing` (vitesse), `animHilbert` (enveloppe), `acos(f2/f1)` (azimut rudimentaire dans `testsFFT.sci`).

6.9 Spectrogrammes : captures mesurées vs modèle théorique

Les figures suivantes montrent les spectrogrammes mesurés des captures F7 du 2019-02-24 (R1/R2, axe vertical en km/h via $f/19.49$, ridge de la balle en cyan, impact marqué), et les captures *générées* théoriques. Le signal modèle (`gen_capture_modele.py`) est une porteuse Doppler à laquelle s'ajoute la modulation de phase du patch $e^{-i(2r/\lambda)\sin\omega t}$, un pic de club à $t_0 = 100$ ms et un bruit de 28 dB. Les captures modèle sont les références de ce que le traitement *devrait* obtenir sur un signal propre – sur le coup modèle de 102 m, le pipe retrouve le spin à −0,1%.

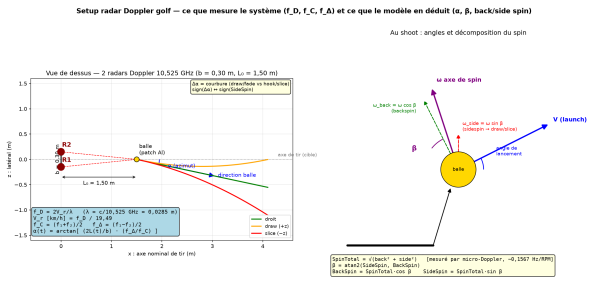


Figure 6.4 – Géométrie du dispositif radar (aussi dans le README).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

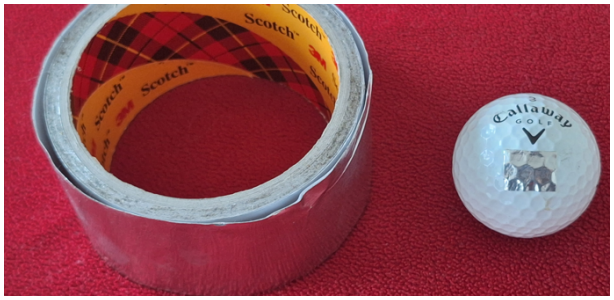


FIGURE 6.5 – Patch sur la balle (ruban aluminium).

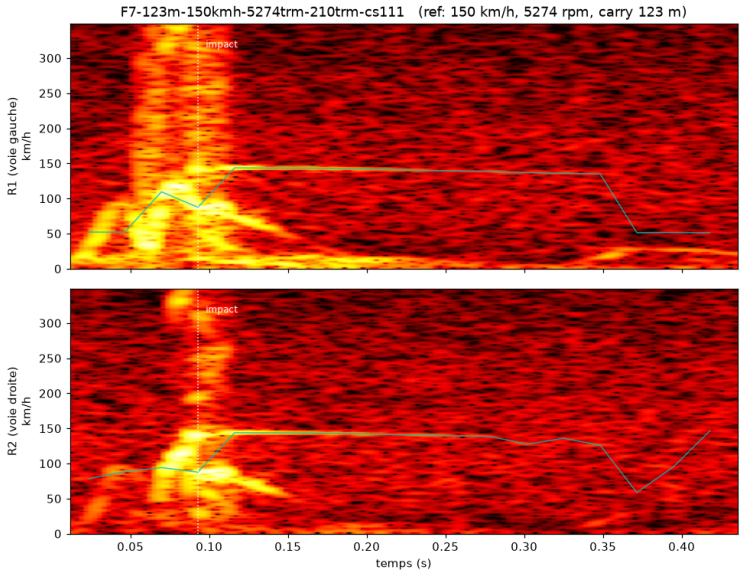


FIGURE 6.6 – Spectrogramme mesuré, coup 123 m / 150 km/h / 5274 tr/min (référence SkyTrak).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

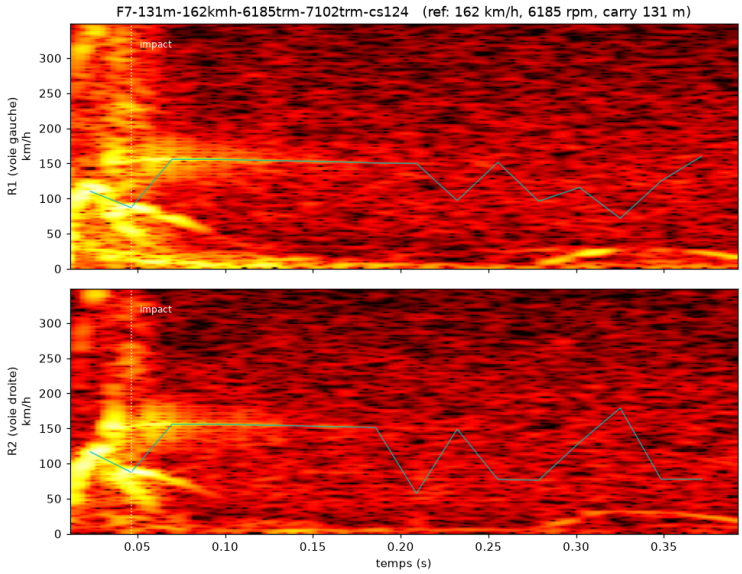


FIGURE 6.7 – Spectrogramme mesuré, coup 131 m / 162 km/h / 6185 back + 7102 latéral tr/min ($\beta \approx 48{,}9°$, forte courbure).

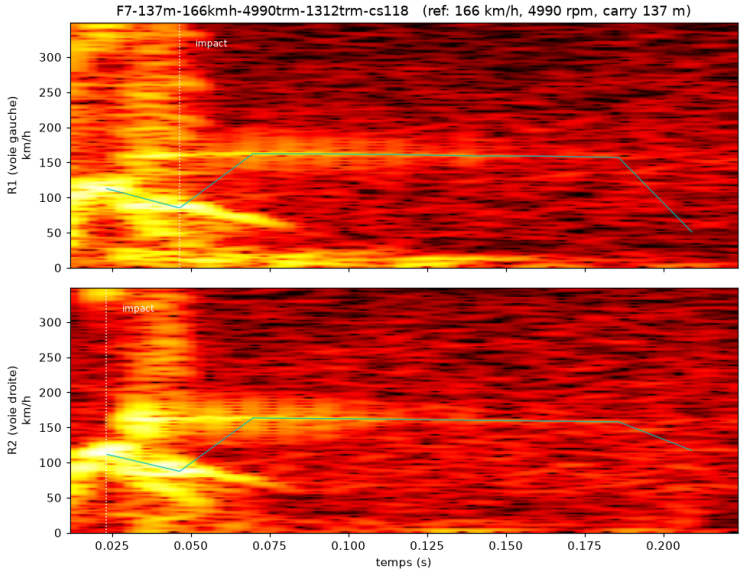


FIGURE 6.8 – Spectrogramme mesuré, coup 137 m / 166 km/h / 4990+1312 tr/min ($\beta \approx 14{,}7°$).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

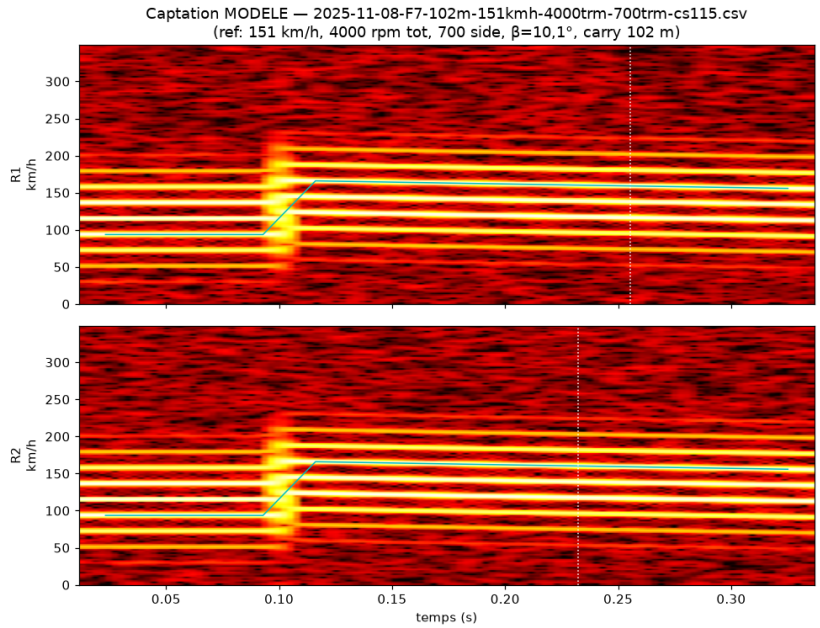


FIGURE 6.9 – Capture théorique (générée), config F7 151 km/h, départ 16,5°, spin 4000 tr/min, latéral 700 tr/min ($\beta = 10{,}1°$), club 115 km/h, portée 102 m.

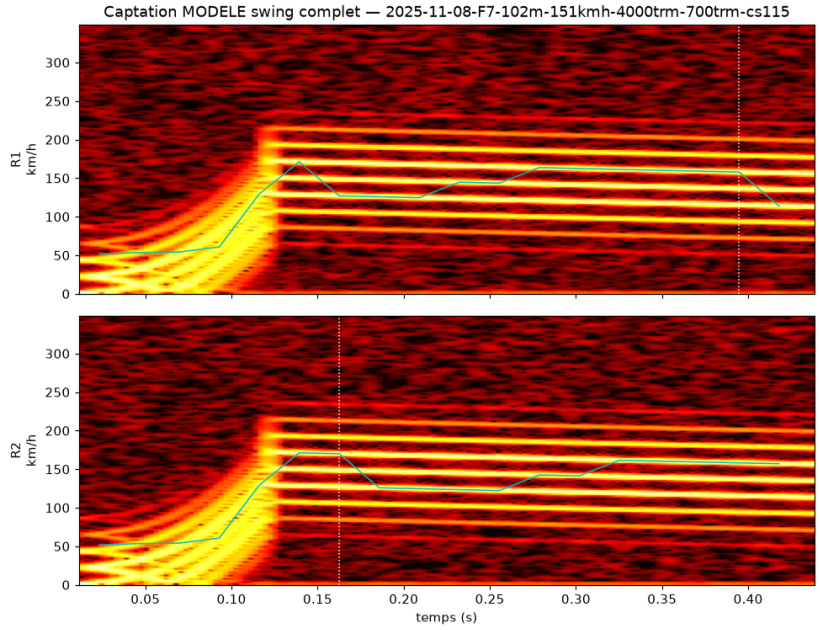


FIGURE 6.10 – Capture générée, même config, version swing complet 450 ms (v2).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

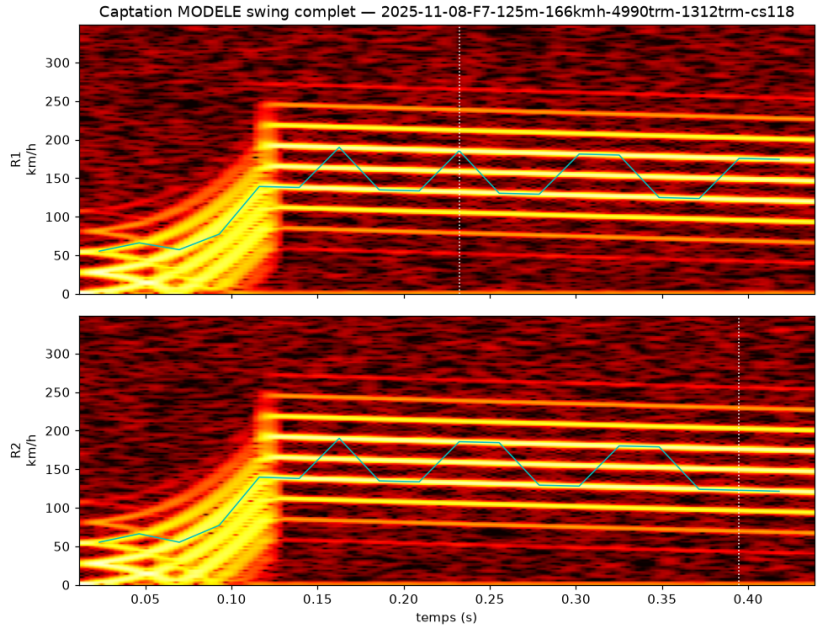


FIGURE 6.11 – Capture générée avec les paramètres du vrai coup de 137 m (166 km/h, 4990/1312 tr/min, club 118).

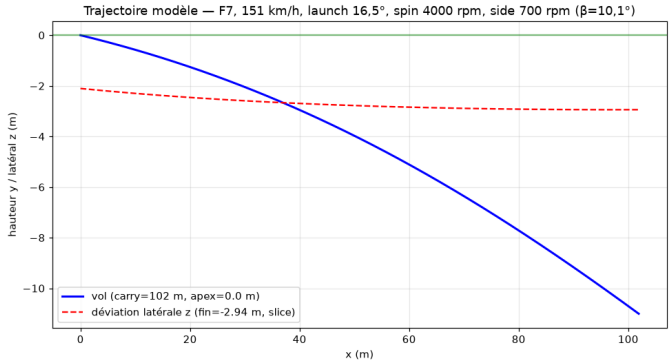


FIGURE 6.12 – Trajectoire modèle (vol + déviation latérale du slice) pour la capture générée du 2025-11-08.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapitre 7

Listings

7.1 Golfball.sci

```
1 // Copyright (C) 2016 - Corporation - Author
2 //
3 //
4 // Date of creation: 16 juin 2016
5 //
6 //SCI2C: DEFAULT_PRECISION= DOUBLE
7 //
8 // ReadWave : FFT_Mat.sce
9 //
```

Listing 7.1 – Listing de la simulation de vol de la balle de golf

```
//
```

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.

Remerciements

Une première version de ce projet a été écrite en 2016 puis laissée de côté. Elle a été reprise en 2026 à la lumière de la puissance des outils d'IA. J'ai repris le projet pour évaluer et apprendre à utiliser l'IA : trouver des bugs dans le code Scilab historique, porter la chaîne de traitement en Python et améliorer les estimateurs au fil de nos discussions (médiane post-impact robuste plutôt que le max global, périodicité du patch plutôt que l'excursion spectrale seule, appariement stéréo des deux radars, contrôles qualité). Cela a aussi montré que le montage électronique (deux radars Doppler 10,525 GHz, amplificateur, Arduino Nano, carte son USB) était bien faisable et cohérent. La traduction française de ce document a été entièrement réalisée par une IA (Mistral Vibe).

Conflits d'intérêts

Les auteurs déclarent n'avoir aucun conflit d'intérêts.

Ce document et les informations qu'il contient sont la propriété de Matthieu PHILIPPE.
Ils ne doivent être ni copiés ni divulgués à des tiers sans autorisation écrite préalable.